

CODE ANALYSIS ENGINE FOR QUANTUM CLASSICAL CODE ROUTER

Jayasinghe Y L

(IT22103840)

B.Sc. (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

CODE ANALYSIS ENGINE FOR QUANTUM- CLASSICAL CODE ROUTER

Jayasinghe Y L

(IT22103840)

Dissertation submitted in the partial fulfilment of the requirements for
B.Sc. (Hons) Degree in Information Technology Specializing in
Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation¹ does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name : Jayasinghe Y L

Student ID : IT22103840

Signature : SIGNATURE SHOULD BE HERE

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

Signature of the Supervisor:

Date:

Signature of the Co-Supervisor:

Date:

ACKNOWLEDGMENT

I wish to express my sincere gratitude to several individuals whose support and guidance made this research possible. First and foremost, I extend my deepest appreciation to my supervisor Prof. Nuwan Kodagoda and co-supervisor Dr. Kapila Dissanayake for their continuous guidance, constructive feedback, and insightful suggestions throughout every stage of this research. Their expertise and encouragement were invaluable in shaping the direction of the Decision Engine component of the Quantum Classical Code Router.

I also extend my appreciation to my fellow team members Siriwardhana A H L T S, Hettiarachchi S R, and Prasad H G A T for their collaborative efforts in building the broader Nexar Quantum-Classical Code Router system. Without their contributions to the surrounding components, the integration of the Code Analysis Engine would not have been possible.

I am grateful to the Department of Information Technology, Sri Lanka Institute of Information Technology, for providing the academic infrastructure, library resources, and computing facilities that supported this research.

Finally, I thank my family for their patience and continued moral support throughout the duration of this project.

ABSTRACT

The rapid expansion of hybrid quantum-classical computing has introduced a pressing need for intelligent systems capable of automatically characterising, classifying, and analysing source code written across multiple quantum programming languages. This individual research component focuses on the design, development, and evaluation of a Code Analysis Engine (CAE) a critical subsystem within the Nexar Quantum-Classical Code Router (QCCR) a broader research project at SLIIT.

The CAE was developed as a Python-based FastAPI microservice that integrates three distinct machine learning-driven analytical pipelines. The first is a multi-model ensemble language classifier that combines CodeBERT, Random Forest, Gradient Boosting, and XGBoost to accurately identify the quantum programming language or framework — namely Cirq, OpenQASM, Python/Qiskit, and Q# — from raw source code. The second is a single-algorithm classifier that applies the same ensemble strategy to detect canonical quantum algorithms such as Grover's search, QFT, Shor's algorithm, VQE, and QAOA from within a given code snippet. The third was an experimental multi-label algorithm classifier leveraging a fine-tuned CodeBERT model to detect combinations of quantum algorithms present simultaneously within a single piece of code, a direction which ultimately revealed important dataset limitations.

The system also incorporates classical complexity analysis through Python's AST module, computing cyclomatic complexity, line and function counts, and control flow metrics. Quantum-specific analysis is performed through a structured pipeline that detects quantum gates, estimates circuit depth, qubit counts, and entanglement characteristics. An Intermediate Representation approach was explored as an enhancement to improve the accuracy and semantic richness of quantum analysis. The engine is deployed as a RESTful API integrated into the broader Nexar system, and is accompanied by a React-based frontend that provides real-time language detection and analysis result visualisation.

Evaluation results demonstrate a peak ensemble language classification accuracy of 98.52% across 1,080 test samples spanning five classes. The single-algorithm ensemble classifier achieved an overall accuracy of 86.39% across 8,800 test samples. The multi-label CodeBERT experiment reached a best F1-micro of 0.3976 due to severe dataset imbalance, which was identified as a key limitation and learning outcome.

Keywords: Code Analysis Engine, Quantum Computing, Machine Learning, Language Classification, Algorithm Recognition, Ensemble Learning, CodeBERT, Abstract Syntax Tree, Hybrid Computing

TABLE OF CONTENTS

DECLARATION	I
ACKNOWLEDGMENT.....	II
ABSTRACT	III
TABLE OF CONTENTS	IV
LIST OF TABLES.....	V
LIST OF FIGURES.....	VI
LIST OF ABBREVIATIONS	VII
1 INTRODUCTION	8
1.1 BACKGROUND AND LITERATURE SURVEY	9
1.2 RESEARCH GAP.....	13
1.3 RESEARCH PROBLEM	14
1.4 RESEARCH OBJECTIVES	15
1.4.1 Main Objective	15
1.4.2 Specific Objectives	15
2 METHODOLOGY	17
2.1 UNDERSTANDING KEY PILLARS OF THE RESEARCH DOMAIN	17
2.1.1 Ensemble Learning.....	17
2.1.2 Transformer-Based Pre-Trained Models (CodeBERT).....	17
2.1.3 Gradient Boosting and Tree-Based Methods.....	18
2.1.4 Abstract Syntax Tree Analysis	18
2.2 APPROACH.....	19
2.2.1 System Architecture	21
2.2.2 Data Collection and Dataset Construction	21
2.2.3 Language Classification Pipeline.....	22
2.2.4 Single-Algorithm Classification Pipeline.....	24
2.2.5 Multi-Label Algorithm Classification — Experimental Attempt.....	26
2.2.6 Classical Complexity Analysis.....	28
2.2.7 Quantum Analysis Pipeline	28
2.2.8 Intermediate Representation Enhancement	30
2.2.9 API Integration and Frontend	32
2.3 SUMMARY OF THE METHODOLOGY	33
2.4 HIGH-LEVEL SYSTEM ARCHITECTURE DIAGRAM	34
2.5 PROJECT REQUIREMENTS THAT HAVE BEEN ACHIEVED.....	34
2.5.1 Functional Requirements.....	34
2.5.2 Non-Functional Requirements.....	35
2.5.3 Other Requirements.....	35
2.6 CONSIDERATION OF ASPECTS OF THE SYSTEM	36
2.6.1 Social Aspects.....	36
2.6.2 Security Aspects.....	36
2.6.3 Ethical Aspects	36
2.6.4 Limitations.....	37
2.7 COMMERCIALISATION ASPECTS OF THE PRODUCT	38
2.7.1 Target Audience.....	38
2.7.2 Market Space.....	38
2.7.3 Revenue Earning	38
2.8 TESTING AND IMPLEMENTATION	38

2.8.1	<i>Code Implementation of the Research Part</i>	38
2.8.2	<i>Testing</i>	41
2.9	TOOLS AND TECHNOLOGIES	42
3	RESULTS AND DISCUSSION	44
3.1	RESULTS	44
3.1.1	<i>Language Classifier Results</i>	44
3.1.2	<i>Single-Algorithm Classifier Results</i>	47
3.1.3	<i>Multi-Label Algorithm Classifier Results</i>	48
3.2	RESEARCH FINDINGS AND DISCUSSION	49
4	CONCLUSION	52
5	REFERENCES	54

LIST OF TABLES

Table 1	Comparison of Existing Approaches and Proposed Code Analysis Engine	14
Table 2.1	Language Classifier Dataset Distribution	22
Table 2.2	Single-Algorithm Classifier Dataset Distribution	22
Table 2.3	Ensemble Weight Configuration	23
Table 2.4	Multi-Label CodeBERT Hyperparameter Configuration	27
Table 2.5	Functional Requirement Summary	34
Table 2.6	Non-Functional Requirements Summary	35
Table 2.7	Tools and Technologies Used	42
Table 3.1	Language Classifier — Accuracy by Model	44
Table 3.2	Language Classifier — Ensemble Metrics (Test Set)	45
Table 3.3	Language Classifier — Per-Class Classification Report	46
Table 3.4	Single-Algorithm Classifier — Per-Class Classification Report	47
Table 3.5	Multi-Label CodeBERT — Best Epoch Metrics	48
Table 3.6	Multi-Label CodeBERT — Per-Algorithm F1 Scores	49

LIST OF FIGURES

Figure 2.1	Code Analysis Engine Component Architecture Diagram	20
Figure 2.2	Language Classification Pipeline Diagram	22
Figure 2.3	Ensemble Model Weight Configuration Diagram	24
Figure 2.4	Single-Algorithm Classification Pipeline Diagram	25
Figure 2.5	Quantum Analysis Pipeline Architecture	29
Figure 2.6	Canonical IR Pipeline Architecture	31
Figure 2.7	Frontend Interface Screenshot — Language Detection	33
Figure 2.8	Frontend Interface Screenshot — Analysis Results	33
Figure 2.9	Full System Overview Diagram of the Nexar QCCR	34
Figure 3.1	Ensemble Language Classifier — Training Confusion Matrices (All Models)	44
Figure 3.2	Ensemble Language Classifier — Evaluation Confusion Matrix	45
Figure 3.3	Single-Algorithm Classifier — Training Confusion Matrices (All Models)	47
Figure 3.4	Single-Algorithm Classifier — Evaluation Confusion Matrix	48

LIST OF ABBREVIATIONS

Abbreviation Description

API	Application Programming Interface
AST	Abstract Syntax Tree
CAE	Code Analysis Engine
BERT	Bidirectional Encoder Representations from Transformers
CI/CD	Continuous Integration / Continuous Deployment
CNN	Convolutional Neural Network
GPU	Graphics Processing Unit
IR	Intermediate Representation
ML	Machine Learning
NISQ	Noisy Intermediate-Scale Quantum
NL	Natural Language
PL	Programming Language
QAOA	Quantum Approximate Optimisation Algorithm
QEC	Quantum Error Correction
QFT	Quantum Fourier Transform
QPE	Quantum Phase Estimation
QPU	Quantum Processing Unit
QCCR	Quantum-Classical Code Router
REST	Representational State Transfer
SDK	Software Development Kit
TF-IDF	Term Frequency – Inverse Document Frequency
VQE	Variational Quantum Eigensolver

1 INTRODUCTION

The emergence of Noisy Intermediate-Scale Quantum (NISQ) devices has created unprecedented opportunities for the development of hybrid quantum-classical computing applications [1]. These hybrid systems leverage the complementary strengths of both classical processors and quantum hardware, allowing certain computationally intensive tasks such as quantum search, optimisation, and simulation to be offloaded to quantum processors, while the classical components handle orchestration, pre-processing, and post-processing. However, building software infrastructure that can intelligently orchestrate this split remains a largely unsolved engineering challenge.

At the heart of any such intelligent orchestration system is the ability to analyse source code to understand what is in it, what language it is written in, what quantum algorithms it employs, how complex it is both classically and quantumly, and where the boundaries between quantum and classical computation lie. Without automated code analysis, developers are forced to manually decide which parts of their code are suited for quantum execution and which are not a task that is not only tedious but requires deep expertise in both quantum computing and software engineering simultaneously.

This research addresses this challenge by developing the Code Analysis Engine (CAE), a core subsystem of the Nexar Quantum-Classical Code Router (QCCR) a collaborative research project conducted at SLIIT. The broader QCCR system consists of six interdependent microservices: the AI Code Converter, the Code Analysis Engine, the Decision Engine, the Hardware Abstraction Layer, a TypeScript API gateway, and a React-based frontend. The CAE, which this report focuses on, serves as the intelligence layer responsible for characterising submitted code, identifying its language and embedded quantum algorithms, computing complexity metrics, and returning a structured analysis report that downstream components particularly the Decision Engine use to route workloads appropriately.

The CAE was built as a Python FastAPI service and integrates three distinct machine learning-driven classification pipelines, a classical complexity analyser based on Python's native Abstract Syntax Tree (AST) module, and a quantum-specific analysis pipeline capable of detecting gates, estimating circuit depth, and computing qubit counts. The system supports four quantum programming languages and frameworks: Python/Qiskit, Cirq, OpenQASM, and Q#. The engine also includes a frontend integration that provides real-time language detection with confidence scores before a full analysis request is submitted.

In terms of machine learning, the CAE developed two functional classifiers and one experimental classifier. The language classifier uses an ensemble of CodeBERT, Random Forest, Gradient Boosting, and XGBoost models to identify the programming framework of submitted code. The single-algorithm classifier applies the same ensemble strategy to identify a dominant quantum algorithm. The multi-label classifier designed to detect multiple algorithms simultaneously within a single code snippet was

implemented using a fine-tuned CodeBERT model but ultimately demonstrated significant limitations due to dataset imbalance, which is documented as a key finding and limitation of this research.

The significance of this work lies in its combination of classical machine learning and quantum-specific domain knowledge to build a practical, end-to-end code analysis system. Unlike existing tools which are either language-specific or limited to static rule-based analysis, the CAE takes a unified, multi-language, learning-based approach that is able to generalise across different quantum frameworks. The system demonstrates that high-accuracy language classification (98.52%) and acceptable algorithm recognition (86.39%) are achievable with relatively small, synthetically augmented datasets provided that the data construction is disciplined and the ensemble architecture is thoughtfully designed.

1.1 Background and Literature Survey

Static Analysis in Quantum Computing

Static analysis refers to the process of examining software source code without actually executing it, with the goal of identifying structural properties, potential bugs, complexity characteristics, or classifications relevant to the analysis task [6]. In the domain of classical software engineering, static analysis is a mature and well-studied field with a rich ecosystem of tools. However, its application to quantum programming is significantly more recent and considerably less developed.

One of the earliest systematic attempts to apply static analysis specifically to quantum code was QChecker, proposed by Zhao et al. in 2023 [6]. QChecker operates on quantum programs written in Qiskit and performs rule-based checks for common quantum programming errors — such as incorrect qubit initialisation, improper measurement placement, and misuse of entanglement operations. While QChecker demonstrated the feasibility of automated quantum code analysis, it was designed exclusively for one framework and could not generalise across the multi-language landscape of modern quantum development.

ScaffCC, introduced by JavadiAbhari et al. in 2014 [2, 7], was a more foundational effort — a compilation and analysis framework for the Scaffold quantum programming language. ScaffCC established key ideas around quantum compilation pipelines and introduced the concept of analysing quantum programs at a structural level using intermediate representations. However, it was tied to a single language and did not incorporate any machine learning-based inference.

More recent compiler-level work has focused on quantum error correction and circuit optimisation rather than general-purpose analysis. Chamberland et al. [13] demonstrated the construction of high-performance compilers for large-scale quantum error correction, highlighting how sophisticated the analysis of quantum circuits can become at scale. While this work is not directly about code classification, it underscores the growing complexity of quantum software and the need for automated analysis tooling.

The broader landscape of quantum software engineering tools has grown substantially in recent years, with frameworks such as IBM Qiskit [9], Google Cirq, Microsoft Q#, and OpenQASM [10] each providing their own analysis and simulation environments. However, each of these operates in isolation — there is no unified analysis layer that can accept code from any of these frameworks and produce a consistent characterisation.

Abstract Syntax Tree Parsing for Quantum Languages

The Abstract Syntax Tree (AST) is a hierarchical data structure that represents the syntactic structure of source code by abstracting away low-level details such as parentheses and semicolons, while preserving the essential semantic and structural relationships between code elements [8]. ASTs are the standard intermediate representation used in compilers, linters, and code analysis tools across virtually every programming language.

In the context of classical Python code, Python's built-in `ast` module provides a native and highly reliable mechanism for parsing source files into ASTs. This capability forms the backbone of the classical complexity analysis component of the CAE developed in this research. Functions, loops, conditionals, and variable assignments can all be traversed and counted from the AST with high precision.

For quantum-specific languages, AST parsing is considerably more challenging. OpenQASM 3.0, introduced by Cross et al. [10], added significant new constructs including timing directives, pulse control, and gate modifiers — making its grammar substantially more complex to parse correctly. Several browser-based quantum language parser implementations have been explored in the literature [9], demonstrating the feasibility of web-oriented parsing for quantum assembly languages, though these approaches are typically demonstration-oriented rather than production-ready.

The challenge of handling multiple quantum dialects simultaneously is well recognised in the literature. Each framework — Qiskit, Cirq, Q#, and OpenQASM — has fundamentally different syntactic conventions. Qiskit code looks like regular Python with specific library calls; Cirq defines circuits using Python object composition; Q# uses a C#-like syntax with quantum-specific keywords; and OpenQASM is a low-level assembly-style language. A unified parsing framework must therefore either translate all of these into a common intermediate representation or apply language-specific parsers that produce normalised outputs.

The approach adopted in this research uses Python's native AST module for Python-based quantum code (Qiskit and Cirq), and regular expression pattern matching augmented with signature-based detection for OpenQASM and Q#. Additionally, a Canonical Intermediate Representation (IR) enhancement was explored as a means to preserve semantic meaning through the analysis pipeline — an approach that builds on concepts from compiler theory where intermediate representations serve as the bridge between source language and target analysis [7].

Quantum Algorithm Pattern Recognition

Recognising the presence of known quantum algorithms within submitted code is a non-trivial classification problem. Algorithms such as Grover's search, Shor's factoring algorithm, the Quantum Fourier Transform (QFT), Quantum Approximate Optimisation Algorithm (QAOA), and the Variational Quantum Eigensolver (VQE) each have characteristic structural signatures — particular gate sequences, qubit connectivity patterns, and parameterisation strategies that distinguish them from one another.

Rule-based pattern recognition for quantum bugs was explored in Q-PAC by Nayak et al. [11], which demonstrated that common quantum programming errors have identifiable patterns that can be detected automatically. While Q-PAC focused on bug detection rather than algorithm recognition, the underlying insight — that quantum code exhibits detectable structural signatures — is directly relevant to the algorithm classification problem addressed in this research.

Machine learning-based approaches to code pattern recognition are increasingly common in classical software engineering contexts. CodeBERT, introduced by Feng et al. [16], is a transformer-based pre-trained model for programming languages that was trained on bimodal data comprising both natural language and source code from six programming languages. CodeBERT's architecture — based on the RoBERTa transformer — enables it to capture deep semantic representations of code that go well beyond simple token matching. This property made it a natural candidate for quantum code classification, as the semantic patterns associated with different quantum algorithms are often subtle and distributed across the structure of the code rather than being concentrated in a single obvious token or keyword.

The diversity of quantum programming languages that must be supported adds another layer of complexity to the algorithm recognition problem. As documented in the literature, the ecosystem of quantum programming languages includes QCL, Q#, Scaffold, Silq, Qiskit, Cirq, and others [12], each with different syntactic representations of the same underlying quantum operations. A robust algorithm classifier must be able to recognise these implementations regardless of the language used to express them.

Complexity Analysis for Quantum-Classical Hybrid Systems

Computational complexity analysis is a well-established discipline in classical computer science, with metrics such as cyclomatic complexity — introduced by McCabe [see background] providing standardised measures of structural complexity in source code. These metrics have proven valuable in software engineering for assessing maintainability, identifying high-risk code regions, and guiding testing efforts.

Extending complexity analysis to the quantum domain requires entirely new metrics. The key quantum complexity metrics that are relevant to practical routing decisions include circuit depth (the longest dependency chain of gate operations), qubit count (the number of qubits required for execution), gate count (the total number of quantum operations), and entanglement complexity (the degree to which qubits are coupled through multi-qubit gates). Each of these has a direct bearing on whether quantum

hardware can practically execute the given algorithm within the coherence time of available NISQ devices.

The challenge of analysing hybrid systems where classical and quantum code are interleaved has been highlighted as a particular gap in existing tooling. Traditional complexity analysis tools address either classical code or quantum circuits in isolation. The CAE addresses this by computing both classical complexity metrics (cyclomatic complexity, function count, line count, control flow depth) and quantum complexity metrics (gate counts, circuit depth estimation, qubit estimation) in a unified pipeline, and returning both sets of results as part of a single analysis report.

The limitations of pure regex-based quantum analysis which was the initial approach in the CAE's development were identified during the implementation process and are documented in detail in Section 2.2.8. Regex-based gate detection can identify the presence of gate names within code but cannot determine which qubits they operate on, whether they are inside loops (and thus execute multiple times), or how they interact to create entanglement. This limitation motivated the exploration of the Canonical IR enhancement.

Machine Learning in Code Analysis

The application of machine learning to source code analysis has been an active research area in classical software engineering for over a decade. Early work demonstrated that simple feature engineering — such as bag-of-words token representations and TF-IDF weighting — combined with traditional classifiers such as Naive Bayes or Support Vector Machines could achieve reasonable performance on tasks like programming language identification and bug classification.

The advent of deep learning and, more recently, transformer-based architectures, has substantially raised the ceiling of what is achievable. Pre-trained models such as CodeBERT [16] capture long-range semantic dependencies in source code through multi-head self-attention, enabling them to understand not just the individual tokens in a piece of code but the relationships between distant parts of the code — a property that is particularly valuable for quantum algorithm recognition, where the signature of an algorithm may be distributed across many lines.

Ensemble methods which combine multiple base learners to produce a final prediction have a long history of improving classification accuracy over individual models. Random Forests, introduced by Breiman [17], combine many decision trees trained on random subsets of features, reducing variance while maintaining low bias. Gradient Boosting, formalised by Friedman [18], takes a different approach by sequentially training weak learners that correct the residual errors of their predecessors. XGBoost, introduced by Chen and Guestrin [19], further extends gradient boosting with computational efficiency improvements and regularisation. In this research, these three tree-based ensemble methods were combined with CodeBERT to form a weighted meta-ensemble — an architecture that proved highly effective for both the language classification and algorithm recognition tasks.

The application of machine learning to quantum code specifically is a novel contribution of this research. Prior work in quantum software engineering has not systematically explored the use of ensemble machine learning for quantum

programming language identification or quantum algorithm recognition at the level of accuracy demonstrated here.

1.2 Research Gap

Despite the advances described above, several significant gaps remain in the existing literature and tooling landscape for quantum-classical code analysis.

I. No Unified Multi-Language Analysis System

Existing quantum development tools and analysis frameworks are uniformly language specific. Qiskit provides tooling for Python-based quantum code, the Q# Development Kit supports Microsoft's quantum language, and Cirq serves Google's ecosystem. There is no existing system capable of accepting code from any of these frameworks and performing a consistent, unified analysis. This is a critical limitation as real-world quantum development projects increasingly use multiple frameworks depending on the target hardware and the nature of the algorithm being implemented.

II. No ML-Based Quantum Algorithm Pattern Recognition

Most existing tools for quantum code analysis focus on error detection and basic syntactic checking. The recognition of canonical quantum algorithms the ability to automatically determine that a given piece of code implements Grover's search or the QFT, for example through machine learning approaches has not been explored in prior work. This capability is essential for intelligent workload routing, as different algorithms have fundamentally different resource requirements and suitability profiles for different quantum hardware backends.

III. Insufficient Tooling for Hybrid System Complexity

Classical complexity analysis tools do not account for quantum operations, and quantum analysis tools do not consider the classical control flow that surrounds and orchestrates quantum circuits. The CAE addresses this by computing both sets of metrics in a single unified pipeline — something that prior tools have not attempted.

IV. Limited Application of Machine Learning to Quantum Code

The intersection of machine learning and quantum code analysis is essentially unexplored territory. While machine learning has proven enormously valuable in classical code analysis — enabling tasks like automatic bug detection, code completion, and program synthesis — its application to quantum code has been almost entirely absent from the literature. This research makes a direct contribution to closing that gap.

V. No Support for Multi-Label Algorithm Detection

Quantum programs often implement combinations of algorithms rather than a single canonical algorithm in isolation. For example, a program might implement both QPE and QFT as components of a larger computation. The multi-label algorithm detection problem — identifying all algorithms present in a single piece of code — has not been attempted in prior work. This research made an experimental attempt at this problem

using a fine-tuned CodeBERT model, and the results — including the failure mode associated with dataset imbalance — provide important insights and groundwork for future research.

Table 1 provides a structured comparison of the proposed CAE against the closest existing systems in the literature.

Table 1: Comparison of Existing Approaches and Proposed Code Analysis Engine

<i>Tool</i>	<i>Multi-language quantum code analysis</i>	<i>Quantum pattern recognition</i>	<i>Hybrid quantum-classical complexity estimation</i>	<i>ML-enhanced quantum code analysis</i>	<i>Multi-label algorithm detection attempt</i>
<i>QChecker (2023)</i>	No	No	No	No	No
<i>ScaffCC (2014)</i>	No	No	No	No	No
<i>Q-PAC (2023)</i>	No	Yes	No	No	No
<i>High-Performance Compiler for QEC (2022)</i>	No	No	Yes	No	No
<i>ML-based Code Analysis</i>	No	No	No	Yes	No
<i>Proposed CAE</i>	Yes	Yes	Yes	Yes	Yes (experimental)

1.3 Research Problem

The central problem that this research addresses can be stated as follows:

how can a unified, intelligent code analysis engine be designed to automatically interpret heterogeneous quantum programs, identify their underlying algorithms, analyze their hybrid computational complexity, and produce structured outputs that can be used by downstream systems such as workload routing engines?

This central problem decomposes into three linked sub-problems, each of which the Code Analysis Engine must address explicitly.

Multi-Language Analysis Challenge

The first sub-problem is enabling consistent analysis across multiple quantum programming frameworks. Modern quantum software development is inherently heterogeneous, with developers using frameworks such as Qiskit, Cirq, Q#, and OpenQASM within the same ecosystem. Existing tools are tightly coupled to specific languages and lack interoperability. The system must therefore be capable of parsing, interpreting, and analysing code written in multiple quantum languages while producing standardised and comparable outputs across all supported frameworks.

Quantum Algorithm Recognition Challenge

The second sub-problem is automatically identifying the quantum algorithms implemented within a given codebase. At present, developers must manually inspect code to determine whether it implements algorithms such as Grover’s search or

Quantum Fourier Transform, which is not scalable in large systems or automated pipelines. The challenge lies in building a machine learning-based recognition mechanism that can detect algorithmic patterns from code representations and classify them accurately, even when implementations vary in structure or optimisation.

Multi-Label Algorithm Detection Challenge

The third sub-problem is detecting multiple co-existing algorithms within a single program. Real-world quantum applications frequently combine several algorithms or subroutines, making single-label classification insufficient. This introduces a multi-label classification problem that remains largely unexplored in current tools. The system must be capable of identifying and assigning multiple algorithm labels to a single code sample, including partial or overlapping implementations, while maintaining high accuracy and interpretability.

1.4 Research Objectives

1.4.1 Main Objective

To design and develop a comprehensive Code Analysis Engine (CAE) that automatically inspects, characterises, and classifies source code written in multiple quantum programming languages. The CAE functions as the analytical core of the Nexar Quantum-Classical Code Router, enabling intelligent workload distribution between quantum processing units and classical computing resources by providing accurate, structured characterisation of submitted source code.

1.4.2 Specific Objectives

The main objective is broken down into seven specific objectives (SO1 to SO7), each of which maps to a distinct phase of the implementation and evaluation effort.

1. SO1 - Design a multi-language code parsing and feature extraction framework: Develop an analysis pipeline capable of processing Python/Qiskit, Cirq, OpenQASM, and Q# source code, extracting structural features suitable for both machine learning-based classification and metrics computation.
2. SO2 - Develop and evaluate a multi-model ensemble language classifier: Train and combine CodeBERT, Random Forest, Gradient Boosting, and XGBoost classifiers using a weighted ensemble strategy to identify the quantum programming language or framework of submitted code with high accuracy.
3. SO3 - Develop and evaluate a single-algorithm quantum algorithm classifier: Design and implement a cost analyzer that combines static hardware pricing with execution-time predictions to produce a cost-aware routing input that integrates with the ML and rule-based layers rather than being a separate downstream che Apply the same ensemble architecture to train a classifier capable of recognising which canonical quantum algorithm from a set including Grover, Shor, QFT, QPE, QAOA, VQE, Amplitude Amplification, Bernstein-Vazirani, Deutsch-Jozsa, and Simon's algorithm is implemented in a submitted code snippet.ck.

4. SO4 - Implement and evaluate a multi-label algorithm classifier: Attempt to develop a fine-tuned CodeBERT model capable of detecting multiple quantum algorithms simultaneously within a single piece of code, and document the outcomes including limitations of this approach.
5. SO5 - Implement a quantum and classical complexity analysis pipeline: Develop modules capable of computing cyclomatic complexity and other classical metrics using Python's AST module, and estimating quantum circuit characteristics including gate counts, circuit depth, and qubit requirements through structured parsing and pattern detection.
6. SO6 - Explore the Canonical Intermediate Representation concept: Investigate and partially implement a Canonical IR approach as an enhancement to the regex-based quantum parsing system, with the goal of preserving semantic information through the analysis pipeline.
7. SO7 - Integrate the engine as a RESTful API service: Deploy the CAE as a FastAPI microservice that exposes structured analysis endpoints for consumption by the broader Nexar system, including the Decision Engine and the frontend.

2 METHODOLOGY

This chapter describes how the Code Analysis Engine (CAE) was designed, implemented, and evaluated. It outlines the key theoretical foundations that underpin the component, including ensemble learning, transformer-based pre-trained models, tree-based machine learning techniques, and abstract syntax tree (AST) analysis. It further explains how these methods were integrated to support multi-language quantum code analysis, automated algorithm recognition, and hybrid complexity estimation. The chapter also details the feature extraction strategies, model architecture, and training approaches used in the system, as well as the rationale behind selecting specific algorithms and tools. Finally, it discusses how these techniques collectively enable the CAE to generate structured, interpretable outputs suitable for downstream components such as intelligent workload routing systems.

2.1 Understanding Key Pillars of the Research Domain

2.1.1 Ensemble Learning

Ensemble learning is a machine learning paradigm in which multiple individual models are trained and their predictions are combined — either by voting, averaging, or through a learned aggregation mechanism — to produce a final output that is typically more accurate and more robust than any single constituent model [17]. The intuition behind ensemble methods is that different models make different errors on different parts of the input space, and by combining them, these errors partially cancel out.

The ensemble strategy employed in this research for both the language classifier and the single-algorithm classifier is a weighted soft-voting ensemble. Each base model produces a probability distribution over the output classes, and the final prediction is the weighted average of these probability distributions. The weights assigned to each model reflect its validation performance — models with higher individual accuracy contribute more to the ensemble's decision. This approach has been shown to consistently outperform individual models across a wide variety of classification tasks [17, 18, 19].

Three tree-based ensemble methods — Random Forest, Gradient Boosting, and XGBoost — are combined with a transformer-based model (CodeBERT) in this research. The combination of fundamentally different model types (tree-based vs. neural) is deliberate, as these model families tend to make uncorrelated errors, which is the key condition for ensemble improvement to occur.

2.1.2 Transformer-Based Pre-Trained Models (CodeBERT)

BERT — Bidirectional Encoder Representations from Transformers — introduced the concept of large-scale pre-training of deep transformer networks on general language modelling objectives, followed by task-specific fine-tuning [see foundational NLP literature]. CodeBERT, introduced by Feng et al. in 2020 [16], extends this paradigm

specifically to source code. CodeBERT is a bimodal pre-trained model trained on natural language and programming language pairs from six programming languages (Python, Java, JavaScript, PHP, Ruby, and Go) using a combination of masked language modelling (MLM) and replaced token detection (RTD) objectives.

The key contribution of CodeBERT relevant to this research is its ability to capture deep semantic representations of source code through multi-head self-attention across 512-token input sequences. When fine-tuned on a task-specific dataset, CodeBERT's pre-trained representations allow it to generalise effectively even from relatively small labelled datasets — a property that is particularly valuable in the quantum code domain, where large labelled datasets are not publicly available.

In this research, CodeBERT (microsoft/codebert-base from HuggingFace) was used in two roles. First, as one of four component models in the language and algorithm ensemble classifiers. Second, as the sole model in the multi-label algorithm classification experiment, where it was fine-tuned using a multi-label binary cross-entropy objective with a sigmoid output layer.

2.1.3 Gradient Boosting and Tree-Based Methods

Random Forest [17] is an ensemble method that builds a large collection of decision trees, each trained on a random bootstrap sample of the training data and using a random subset of features at each split. Predictions are aggregated by majority voting (classification) or averaging (regression). Random Forest is known for its robustness to overfitting, its ability to handle high-dimensional feature spaces, and its interpretability through feature importance scores.

Gradient Boosting [18] takes a sequential approach, where each new tree is trained to correct the residual errors of all preceding trees. This is formalised as gradient descent in function space, where the loss is minimised iteratively. Gradient Boosting tends to produce lower bias than Random Forest at the cost of requiring more careful hyperparameter tuning and being more prone to overfitting if not regularised.

XGBoost [19] extends Gradient Boosting with several engineering and algorithmic improvements, including second-order gradient approximations, regularisation terms added to the loss function, column subsampling (similar to Random Forest), and efficient handling of sparse inputs. XGBoost has been widely successful in structured data competitions and applied classification tasks, and was selected for inclusion in the ensemble based on its complementary characteristics relative to standard Gradient Boosting.

For both the language and algorithm classifiers, features fed into the tree-based models were derived from a combination of TF-IDF-weighted token frequency vectors, handcrafted structural features (such as keyword occurrence counts), and optionally, CodeBERT embeddings — ensuring that these models had access to meaningful information about the code's content.

2.1.4 Abstract Syntax Tree Analysis

The Abstract Syntax Tree (AST) is a tree-based representation of source code that captures its syntactic structure while abstracting away irrelevant details such as

whitespace, comments, and punctuation [8]. In Python, the built-in `ast` module provides a native, production-ready parser that produces AST representations of Python source code. These representations expose the hierarchical structure of the code — functions, classes, loops, conditionals, assignments — in a form that is easy to traverse and analyse programmatically.

In the CAE, the Python AST module is used for classical complexity analysis. A custom AST visitor traverses the parsed tree and collects metrics including the total number of lines, the number of function definitions, the number of class definitions, the number of import statements, and a cyclomatic complexity estimate based on the number of branching constructs (if statements, for loops, while loops, try-except blocks) encountered. This provides a structured, reliable measure of the classical complexity of the submitted code regardless of whether it also contains quantum operations.

For quantum code written in Python-based frameworks (Qiskit and Cirq), the same AST analysis is applicable because these frameworks express quantum circuits within regular Python code. For OpenQASM and Q#, which have their own non-Python syntaxes, the AST-based analysis is not directly applicable, and signature-based pattern matching is used instead.

2.2 Approach

The Code Analysis Engine was developed as an iterative, pipeline-oriented system, with each analytical capability built as a modular component that could be independently developed, tested, and integrated. The development process followed an agile approach, with functionality progressively added and refined based on evaluation results. Three main machine learning components were built: the language classifier, the single-algorithm classifier, and the experimental multi-label classifier. These were complemented by a classical complexity analysis module and a quantum analysis pipeline, all integrated through a FastAPI-based REST interface.

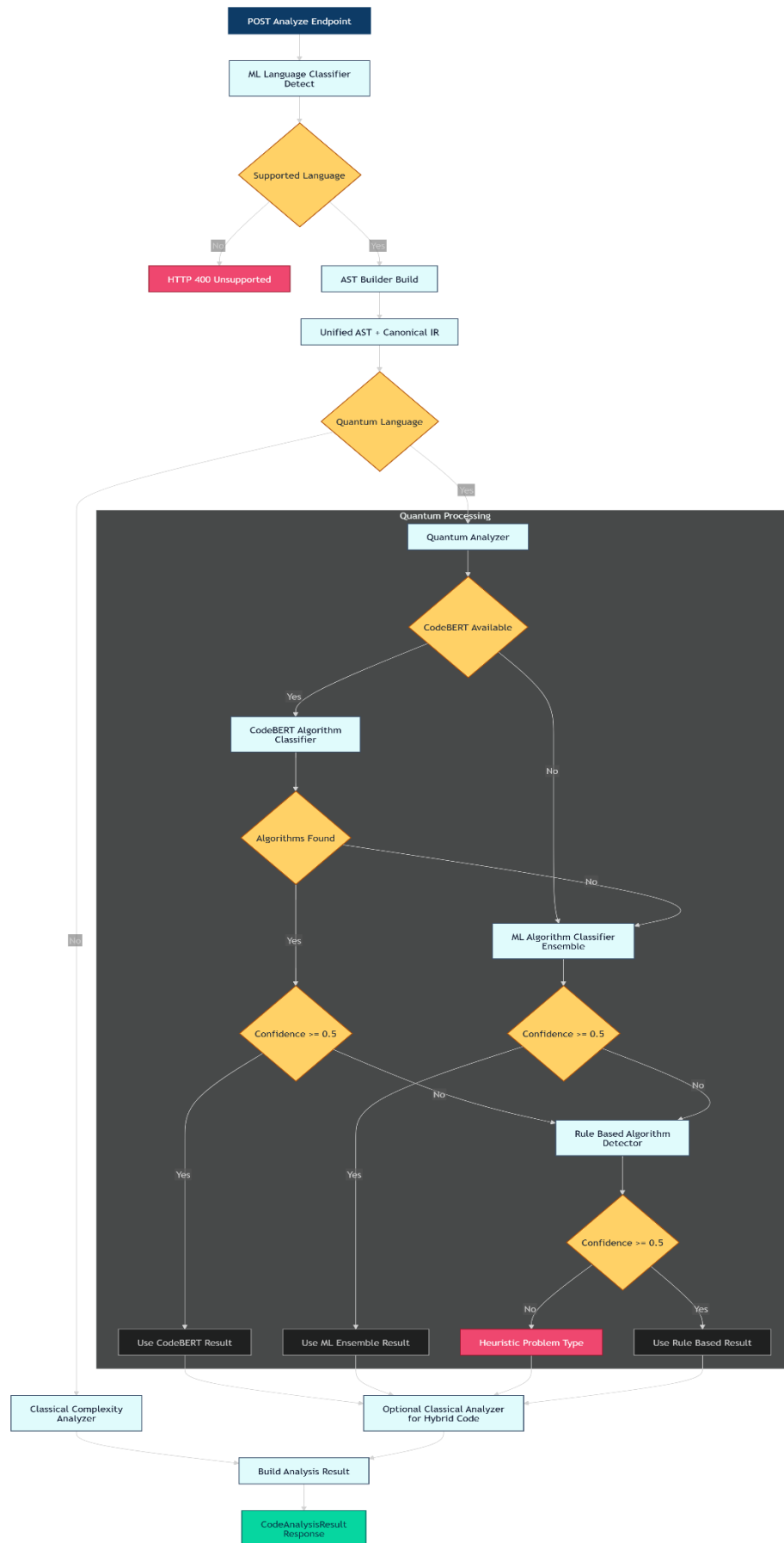


Figure 2.1: Code Analysis Engine Component Architecture Diagram

2.2.1 System Architecture

The CAE operates as a standalone Python FastAPI microservice exposed on port 8002, within the broader six-service Nexar QCCR system. When a code analysis request arrives at the CAE's primary /analyse endpoint, the system executes the following sequential pipeline.

The incoming source code is first passed to the language detection module, which uses the trained ensemble language classifier to determine which quantum programming framework the code belongs to. This classification is performed using the weighted soft-voting ensemble of CodeBERT, Random Forest, Gradient Boosting, and XGBoost models. The predicted language label and associated confidence score are included in the response.

Concurrently, the code is passed to the classical complexity analyser, which uses the Python ast module (for Python-based code) or structured token counting (for Q# and OpenQASM) to compute classical software metrics including cyclomatic complexity, line count, function count, import count, and control flow depth.

The code is then passed to the quantum analysis pipeline. This component uses a combination of framework-specific parsers and regular expression-based gate detection to identify quantum gates, estimate circuit depth using a dependency graph (critical path) approach, estimate qubit counts from register declarations, and assess the presence of entanglement patterns.

Simultaneously, the code is passed to the single-algorithm classifier, which uses the ensemble model to predict which canonical quantum algorithm the code most likely implements, along with a confidence score.

The results from all four sub-components are aggregated into a structured JSON response that is returned to the caller. Downstream components in the Nexar system particularly the Decision Engine consume this response to make workload routing decisions.

2.2.2 Data Collection and Dataset Construction

One of the most significant challenges in this research was the lack of publicly available, labelled datasets of quantum code. No existing benchmark dataset existed that covered all four supported quantum frameworks and all ten quantum algorithms in a form suitable for machine learning. All training data used in this research was therefore either synthetically generated, collected from open-source repositories on GitHub, or manually authored.

Language Classifier Dataset

The language classification dataset was constructed by collecting code samples from four classes: Cirq, OpenQASM, Python/Qiskit, and Q#. A fifth class "python" was maintained as a separate category to distinguish generic Python code from Qiskit-specific quantum Python code. Samples were sourced from GitHub repositories, Qiskit tutorials, Cirq documentation examples, OpenQASM circuit libraries, and Q# Quantum Development Kit samples. The dataset was balanced to ensure reasonable representation of all five classes. The total dataset comprised 3,600 training samples and 1,080 test samples across five classes.

Table 2.1: Language Classifier Dataset Distribution

Class	Training Samples	Test Samples
Cirq	~900	273
OpenQASM	~750	229
Python	~675	206
Qiskit	~750	220
Q#	~525	152

Single-Algorithm Classifier Dataset

The single-algorithm classification dataset was constructed across ten quantum algorithm classes: Amplitude Amplification, Bernstein-Vazirani, Deutsch-Jozsa, Grover's Search, QAOA, QFT, QPE, Shor's Algorithm, Simon's Algorithm, and VQE. Algorithm implementations were manually authored, adapted from textbook sources, and collected from open-source quantum libraries. The dataset was intentionally balanced with 400–1,600 samples per class depending on the natural availability of implementations. The total dataset comprised approximately 26,400 training samples and 8,800 test samples.

Table 2.2: Single-Algorithm Classifier Dataset Distribution

Algorithm	Training Samples	Test Samples
Amplitude Amplification	1,200	400
Bernstein-Vazirani	4,800	1,600
Deutsch-Jozsa	4,800	1,600
Grover	4,800	1,600
QAOA	1,200	400
QFT	4,800	1,600
QPE	1,200	400
Shor	1,200	400
Simon	1,200	400
VQE	1,200	400

Multi-Label Algorithm Classifier Dataset

The multi-label dataset was constructed with 2,400 total samples across 8 algorithm classes, with distributions ranging from 150 to 600 samples per class. This severe imbalance a direct consequence of the fact that not all quantum programming frameworks provide canonical implementations of all algorithms proved to be a critical weakness and is discussed in detail in Section 2.2.5.

2.2.3 Language Classification Pipeline

The language classifier was developed to identify the quantum programming framework Cirq, OpenQASM, Python (generic), Qiskit, or Q# from submitted source code. The classifier architecture employs a weighted soft-voting ensemble of four base models: CodeBERT, Random Forest, Gradient Boosting, and XGBoost.

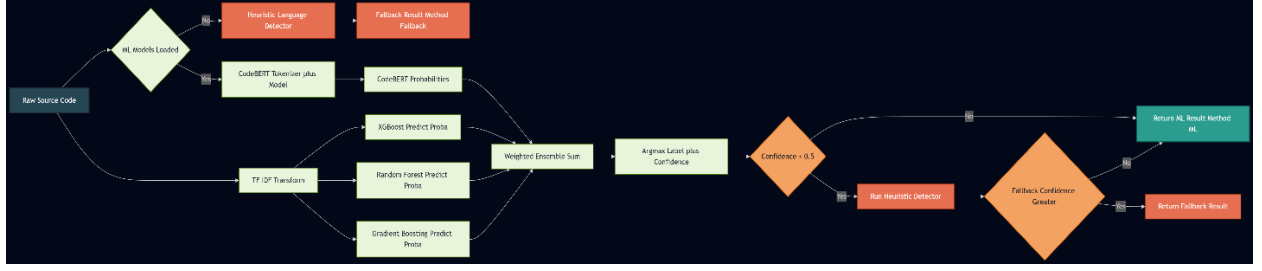


Figure 2.2: Language Classification Pipeline Diagram

Feature Engineering

For the tree-based models (Random Forest, Gradient Boosting, XGBoost), features were derived from two sources. The first was a TF-IDF vectorised representation of the source code tokens, capturing the relative frequency of different tokens across the corpus. This is particularly effective for language identification because different frameworks use very distinctive vocabulary — Qiskit code contains tokens like `QuantumCircuit`, `qc.h`, and `qc.cx`; Cirq code uses `cirq.LineQubit` and `cirq.Circuit`; Q# uses `operation`, `using`, and `Qubit`; and OpenQASM contains directives like `qreg`, `creg`, and `OPENQASM`. The second feature source was a set of handcrafted signature scores computed by counting the occurrence of framework-specific keywords and patterns — effectively a scoring system with over 40 signatures per language.

For CodeBERT, the full source code (truncated to 512 tokens if necessary) was tokenised using the `microsoft/codebert-base` tokeniser, and the `[CLS]` token embedding from the final hidden layer was used as the feature representation. This 768-dimensional embedding captures deep semantic and syntactic properties of the code that go beyond surface-level token frequency.

Model Training

Each of the four base models was trained independently on the same training dataset. Random Forest was trained with 200 estimators. Gradient Boosting used 100 estimators with a learning rate of 0.1. XGBoost was trained with 200 rounds and early stopping. CodeBERT was fine-tuned for 5 epochs with a batch size of 16 and a learning rate of $2e-5$, with the linear classification head added on top of the `[CLS]` embedding.

Ensemble Weight Configuration

The ensemble weights were determined empirically based on validation set performance of each individual model. The final weights assigned were CodeBERT: 0.30, XGBoost: 0.20, Random Forest: 0.25, Gradient Boosting: 0.25. These weights reflect CodeBERT's superior individual performance as a transformer model while ensuring that the tree-based models contribute meaningfully to the final prediction.

Table 2.3: Ensemble Weight Configuration

Model	Ensemble Weight
CodeBERT	0.30
XGBoost	0.20
Random Forest	0.25
Gradient Boosting	0.25

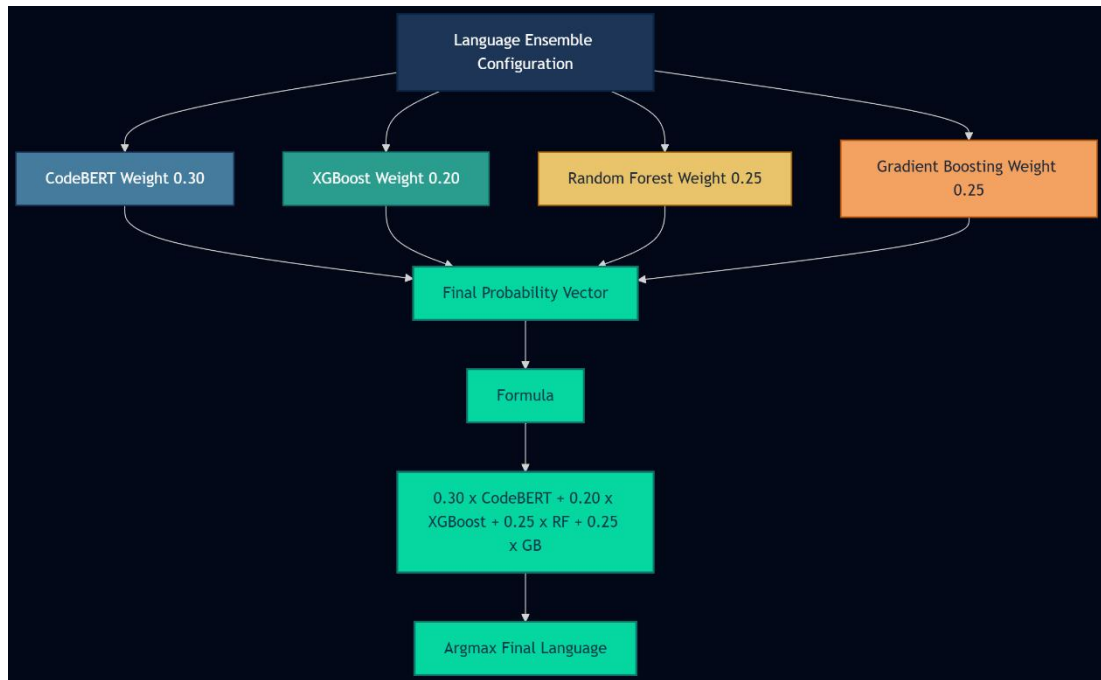


Figure 2.3: Ensemble Model Weight Configuration Diagram

Prediction Process

At inference time, the submitted code is simultaneously processed by all four models. Each model produces a probability vector over the five language classes. These vectors are scaled by the corresponding weights and summed to produce a final probability distribution. The class with the highest probability in this weighted sum is the predicted language label. The confidence score returned to the user is the maximum probability in the final weighted distribution.

2.2.4 Single-Algorithm Classification Pipeline

The single-algorithm classifier was built using the same ensemble architecture as the language classifier but trained on the ten-class quantum algorithm dataset. Its purpose is to determine which canonical quantum algorithm is most prominently implemented in a submitted piece of code.

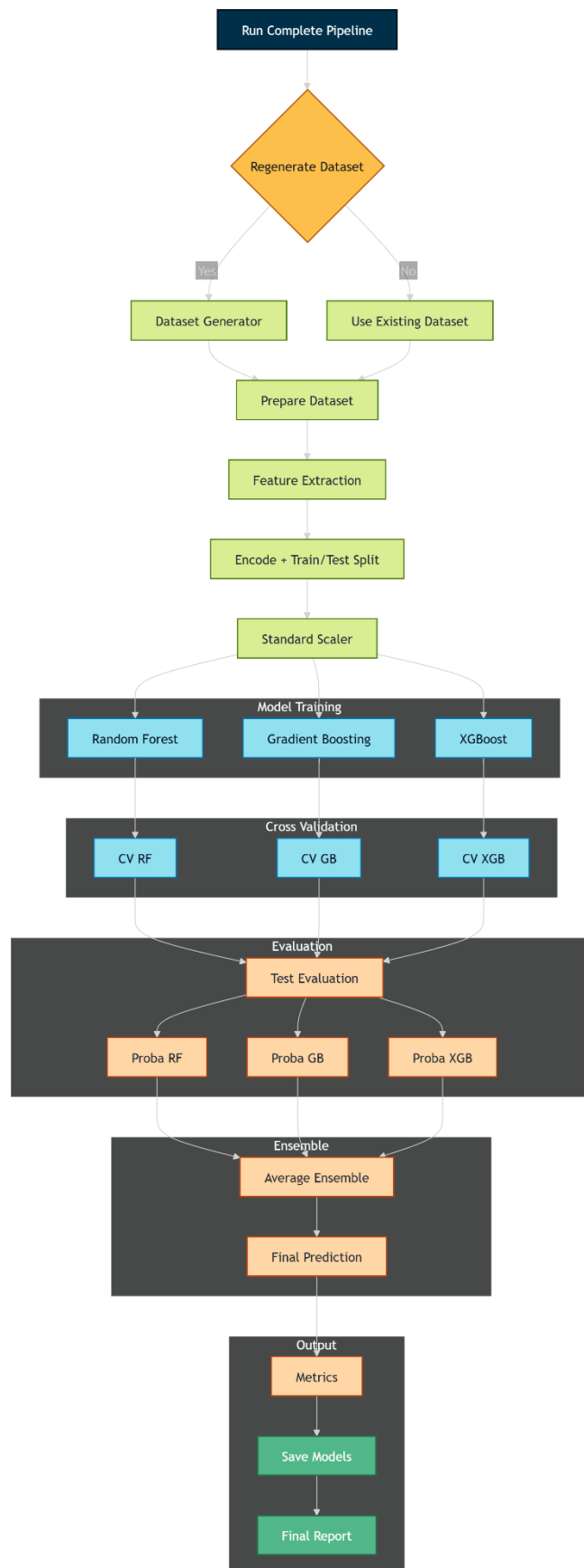


Figure 2.4: Single-Algorithm Classification Pipeline Diagram

The feature engineering approach mirrors that of the language classifier: TF-IDF token frequency vectors combined with handcrafted structural features for the tree-based models, and CodeBERT [CLS] embeddings for the neural model. Algorithm-specific signatures were also constructed for example, Grover's algorithm implementations tend to feature oracle constructions, amplitude amplification loops, and characteristic patterns of Hadamard gates applied to all qubits; QFT implementations feature cascading controlled rotation gates; VQE implementations contain parameterised variational ansatz circuits with classical optimiser loops.

The training procedure was identical to the language classifier, with the same model architectures and similar hyperparameters adjusted for the larger dataset size. The same weighted soft-voting ensemble strategy was applied at inference time, producing a probability distribution over ten algorithm classes.

It is important to note that this classifier is designed for the case where a code snippet implements one dominant algorithm. It is not designed to handle the multi-algorithm case that was the purpose of the separate multi-label classifier experiment described in the next section. The single-algorithm classifier performs best when the submitted code is a focused, self-contained implementation of one particular algorithm.

2.2.5 Multi-Label Algorithm Classification — Experimental Attempt

The multi-label algorithm classifier was conceived to address a more realistic and challenging scenario: quantum programs that implement combinations of algorithms simultaneously. For example, a Shor's algorithm implementation inherently incorporates QFT as a sub-routine; a QAOA implementation may include VQE-like variational components; and many practical quantum programs use common sub-routines like phase kickback or amplitude amplification as components of larger algorithms.

Motivation

The ability to detect all algorithms present in a piece of code rather than just the dominant one would provide richer information to the Decision Engine and would more accurately reflect the structure of real-world quantum programs. This was an aspirational objective that went beyond the core deliverables of the project and was pursued as an extension once the single-label classifiers were complete.

Architecture

The multi-label classifier used a fine-tuned CodeBERT model as its sole component. Rather than a softmax output layer (which produces a probability distribution over mutually exclusive classes), a sigmoid output layer was used with one output neuron per class. Binary cross-entropy loss was applied independently to each output neuron, allowing the model to predict multiple positive classes simultaneously. The threshold for a positive prediction was set at 0.5 across all classes.

The dataset constructed for this classifier contained 2,400 samples across 8 algorithm classes (excluding Amplitude Amplification and Simon's Algorithm due to insufficient multi-algorithm samples). The label distribution was severely

imbalanced, with Bernstein-Vazirani having 600 samples, while QPE and QAOA had only 150 each.

Training

The model was trained for 15 epochs with a batch size of 8 and a learning rate of $2e-5$ with a maximum sequence length of 512 tokens. The best performance was recorded at epoch 14.

Table 2.4: Multi-Label CodeBERT Hyperparameter Configuration here

Hyperparameter	Value
Epochs	15
Batch Size	8
Learning Rate	$2e-05$
Max Sequence Length	512
Classification Threshold	0.5

Outcomes and Failure Analysis

The multi-label classifier produced a best F1-micro of 0.3976, an exact match accuracy of 0.40, and a Hamming loss of 0.1515. Per-algorithm F1 scores varied dramatically: Grover, QAOA, QPE, Shor, and VQE all achieved F1 scores of 1.0, while Bernstein-Vazirani and Deutsch-Jozsa achieved 0.0, and Simon achieved 0.469. This extreme variance is a clear signature of the impact of dataset imbalance. The model learned to predict the well-represented classes accurately but failed entirely on the under-represented ones.

The root cause was identified as a fundamental dataset limitation: not all quantum algorithms have canonical implementations across all four supported quantum frameworks. OpenQASM, for example, is a low-level assembly language that is rarely used to implement high-level algorithms like VQE or QAOA from scratch. Q# has limited publicly available implementations of Bernstein-Vazirani and Simon's Algorithm compared to Qiskit. This meant that constructing a balanced multi-label dataset was extremely difficult without resorting to synthetic data generation of such poor quality that it would harm the model rather than help it.

This failure is documented honestly and thoroughly in this report because it represents a genuine learning outcome of the research. The inability to build a reliable multi-label quantum algorithm classifier, given the current state of open-source quantum code availability, is itself an important finding that future researchers in this space need to be aware of. The problem is not with the model architecture — CodeBERT is well-suited to this task in principle — but with the data. Future work in this area will require either much larger datasets collected from production quantum code repositories, or carefully designed synthetic data augmentation strategies that preserve the semantic properties of quantum algorithm implementations.

2.2.6 Classical Complexity Analysis

Classical complexity analysis within the CAE is performed using Python's built-in `ast` module, which provides a reliable and robust mechanism for parsing Python source code into a traversable Abstract Syntax Tree. This approach is applicable to all Python-based quantum code (Qiskit and Cirq), as these frameworks express quantum circuits within regular Python programs.

A custom `ComplexityAnalyser` visitor class traverses the parsed AST and collects the following metrics. Line of code count is determined from the source text. Function definition count is collected by traversing `ast.FunctionDef` nodes. Class definition count is collected from `ast.ClassDef` nodes. Import count is collected from `ast.Import` and `ast.ImportFrom` nodes. Cyclomatic complexity is estimated by counting the number of branching constructs: each `ast.If`, `ast.For`, `ast.While`, `ast.Try`, and `ast.ExceptHandler` node increments the cyclomatic complexity counter by one. The base complexity for a function is 1, so a function with no branches has cyclomatic complexity 1, and each additional branch adds 1.

For Q# and OpenQASM code, which cannot be parsed by Python's AST module, a simplified line-counting and keyword-frequency approach is used to produce approximate classical complexity estimates.

The complexity metrics produced by this module are included in the analysis response under a `classical_complexity` field, providing the Decision Engine with information about the structural complexity of the code from a classical software engineering perspective. High cyclomatic complexity, for example, may indicate that the code has complex classical control flow that is not well-suited to quantum execution even if it contains some quantum operations.

2.2.7 Quantum Analysis Pipeline

The quantum analysis pipeline is responsible for extracting quantum-specific metrics from submitted source code. This is implemented as a set of framework-specific parsers and analysers, each targeting one of the four supported quantum frameworks.

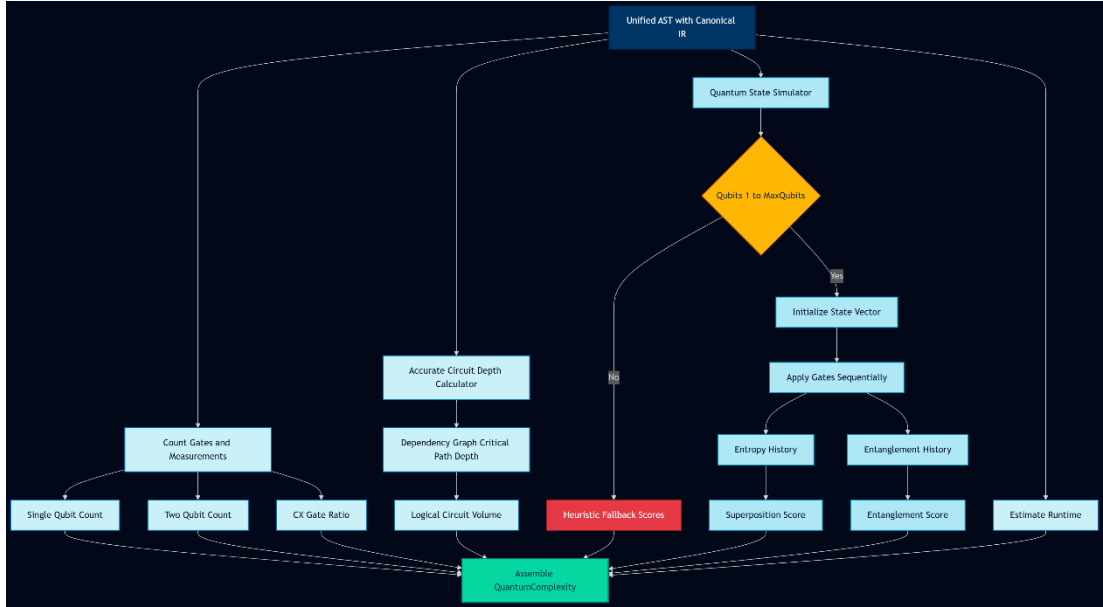


Figure 2.5: Quantum Analysis Pipeline Architecture

Gate Detection

Gate detection is performed using a combination of Python AST analysis (for Qiskit and Cirq code) and regular expression pattern matching (for all four frameworks). For Qiskit code, the AST is traversed to identify method call expressions on `QuantumCircuit` objects, with the method name compared against a comprehensive gate name database that includes standard gates (H, X, Y, Z, CNOT, CZ, CX, T, S, Rz, Ry, Rx, CCX/Toffoli, SWAP), controlled operations, and framework-specific gates. For Cirq, the pattern involves looking for gate application expressions such as `cirq.H(qubit)` and `cirq.CNOT(q0, q1)`. For OpenQASM, standard assembly mnemonics are matched. For Q#, operation calls from the `Microsoft.Quantum` namespace are identified.

The gate detector produces a count of detected gates by type, a total gate count, and a list of detected gate types present in the code.

Qubit Estimation

Qubit estimation for Qiskit code is performed by identifying `QuantumCircuit` constructor calls and extracting the integer arguments, which specify the number of qubits. For Cirq, `LineQubit.range(n)` and `NamedQubit` declarations are identified. For OpenQASM, `qreg q[n]` declarations are parsed. For Q#, use `q = Qubit[n]` allocations are identified. In cases where the qubit count is expressed as a variable rather than a literal integer — which occurs frequently in parameterised or loop-based circuit constructions — the system falls back to counting the number of distinct qubit references, which provides an approximation.

Circuit Depth Estimation

Circuit depth the length of the critical path through the circuit — is estimated using a dependency graph approach. Detected gates are modelled as nodes in a directed

acyclic graph, with edges representing data dependencies (a gate that uses qubit q as input depends on all previous gates that used qubit q as output). The critical path length through this graph gives the circuit depth. This is necessarily an approximation for dynamically constructed circuits where gate sequences are determined at runtime, but provides a useful estimate for statically constructed circuits.

Entanglement Detection

The presence of entanglement is assessed by checking for the presence of multi-qubit gates in the detected gate list. If CNOT, CZ, CX, Toffoli, or SWAP gates are detected, the circuit is flagged as potentially entangling. A more nuanced metric the entanglement ratio is computed as the fraction of all detected gates that are multi-qubit gates, which provides a rough measure of the degree of entanglement in the circuit.

Superposition Detection

The presence of superposition is assessed by checking for the presence of Hadamard (H) gates or other gates that create superposition states (such as certain rotation gates). This is reported as a boolean flag indicating whether superposition is created.

It is important to acknowledge the limitations of this approach. As discussed in the implementation overview documentation, regex-based and simple AST-based gate detection is inherently limited in its ability to capture semantic meaning. A gate detected in the code may not actually create entanglement if the qubits it acts on are not connected through a data flow path. Loop unrolling the recognition that a gate inside a `for` loop of n iterations actually executes n times, not once — is not fully accounted for in the current implementation. These limitations are what motivated the Canonical IR enhancement described in the next section.

2.2.8 Intermediate Representation Enhancement

During the development of the quantum analysis pipeline, a significant limitation of the initial approach was identified: the combination of regex-based gate detection and a flat list-based unified AST representation loses critical information about the semantic meaning of detected quantum operations. Two specific failure modes were identified and documented.

The Unified AST Information Loss Problem

The initial intermediate representation called the Unified AST stored detected quantum operations as a flat list of objects, each recording the gate type and line number but not the qubit indices, the surrounding control flow context, or the data dependencies between operations. As a result, a gate detected inside a loop appears identical to the same gate detected outside a loop, even though its actual execution count may differ by orders of magnitude. Similarly, the qubit indices of dynamically computed gates (where the qubit is specified as $i \% n$ rather than a literal integer) cannot be resolved without evaluating the surrounding Python code.

The Regex Gate Detection Semantic Gap

Pure regex-based gate detection identifies the name of a gate function call but cannot determine the qubit connectivity which qubit is the control and which is the target or whether the detected gate is actually executed (as opposed to being referenced in a comment or a string literal). More fundamentally, knowing that a CNOT gate is present does not immediately tell you whether entanglement is created, because that depends on whether the control and target qubits were previously in a product state or an entangled state.

The Canonical IR Approach

To address these limitations, a Canonical Intermediate Representation (Canonical IR) was designed and partially implemented. The Canonical IR is a richer semantic representation of a quantum circuit that preserves gate connectivity (explicit control and target qubit identities), data flow dependencies (which gates depend on which preceding gates), execution context (whether a gate is inside a loop, and with what iteration count), and parameter values (resolved as far as statically possible).

The Canonical IR pipeline converts source code into a semantically meaningful representation in three stages. First, an official language-specific parser (from Qiskit's own internal circuit representation, for example) is used to produce a structurally accurate circuit object. Second, this circuit object is converted into the Canonical IR a Python data structure with explicit gate operations, qubit assignments, and dependency edges. Third, metrics are extracted deterministically from the Canonical IR rather than being estimated from the raw source text.

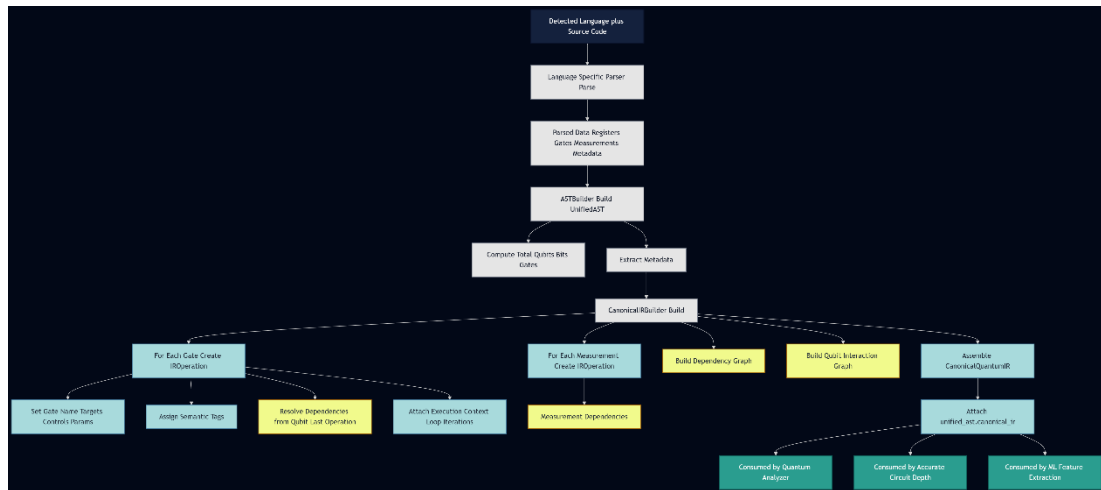


Figure 2.6: Canonical IR Pipeline Architecture

This approach was partially implemented and tested. The Canonical IR successfully resolved the loop unrolling problem for simple cases (fixed-iteration loops) and correctly identified qubit connectivity for standard gate patterns. However, it could not handle all cases: highly parameterised circuits where qubit targets are computed from arbitrary Python expressions, circuits constructed using higher-order functions or class inheritance, and code that builds circuits through dynamic string-based

circuit construction cannot be fully resolved statically. These remaining cases are acknowledged as limitations and represent directions for future work.

The implementation of the Canonical IR concept represents one of the key enhancements made during the development process beyond the initial proposal and demonstrates the research team's understanding of the deeper challenges in semantic quantum code analysis.

2.2.9 API Integration and Frontend

REST API Design

The CAE is exposed as a FastAPI service running on port 8002. The primary analysis endpoint accepts POST requests with a JSON body containing the source code string. The response includes the predicted language with confidence score, the predicted algorithm with confidence score, the classical complexity metrics, the quantum analysis metrics (gate counts, circuit depth, qubit estimate, entanglement flag), and any applicable error messages.

FastAPI was chosen for its native support for asynchronous request handling, its automatic OpenAPI documentation generation, and its clean integration with Pydantic for request and response validation. The service includes CORS middleware to support cross-origin requests from the frontend.

Frontend Integration

The CAE is integrated with a React + Vite frontend served on port 5173. The frontend provides two points of CAE interaction. The first is real-time language detection as the user types or pastes code into the editor, a lightweight signature-based language detection function (implemented in JavaScript on the frontend, mirroring the Python backend logic) immediately displays the detected language with a confidence score. This provides instant visual feedback without requiring a round trip to the backend. The second is the full analysis when the user clicks the "Analyse" button, a POST request is sent to the CAE's analysis endpoint, and the results are displayed in a structured results dashboard showing all computed metrics.

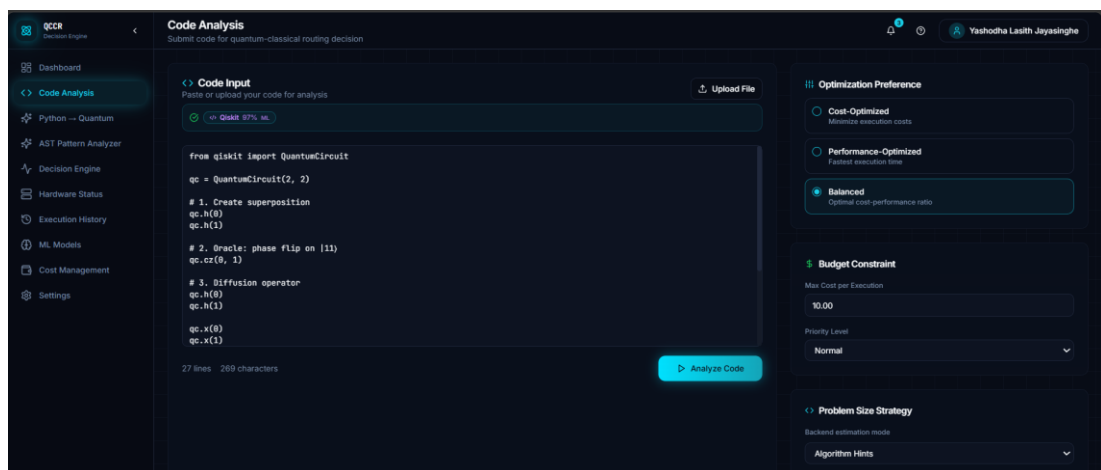


Figure 2.7: Frontend Interface Screenshot — Language Detection

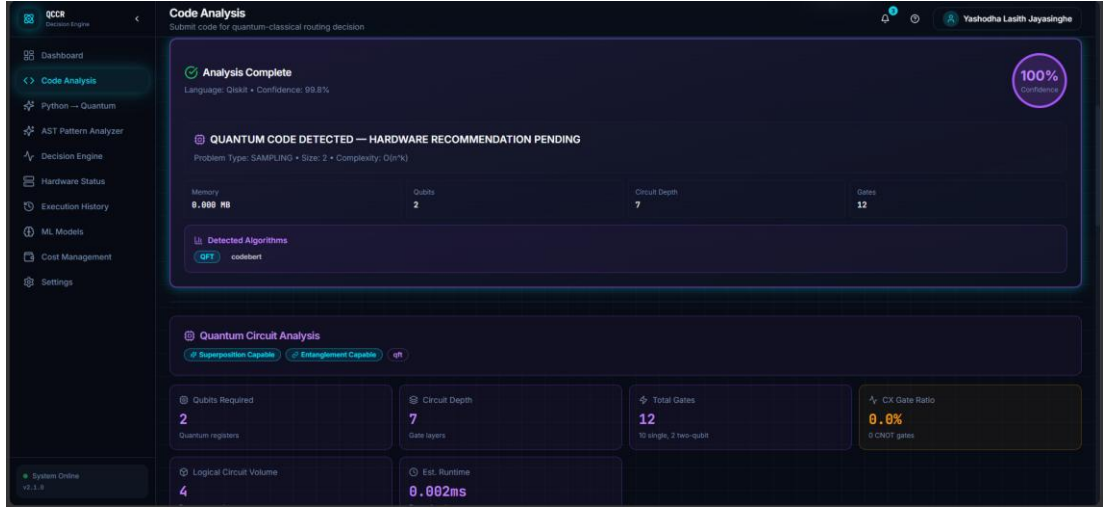


Figure 2.8: Frontend Interface Screenshot — Analysis Results

2.3 Summary of the Methodology

The Code Analysis Engine was developed as a modular, layered system with four primary analytical capabilities: language classification, algorithm classification, classical complexity analysis, and quantum-specific metric extraction. All capabilities were integrated through a FastAPI REST interface and consumed by a React frontend.

The ML classifiers were trained on custom-built datasets of quantum code spanning four frameworks and ten algorithms. An ensemble approach combining CodeBERT, Random Forest, Gradient Boosting, and XGBoost was used for both language and single-algorithm classification, achieving strong results. An experimental multi-label classifier was implemented using CodeBERT alone but revealed important dataset limitations that prevented satisfactory performance.

A key enhancement during development was the exploration and partial implementation of the Canonical IR concept, which addresses fundamental limitations of the initial regex and flat-AST-based quantum analysis approach. This enhancement represents an honest engagement with the challenges of semantic quantum code analysis that goes beyond the scope of the original proposal.

2.4 High-Level System Architecture Diagram

Nexar System Architecture

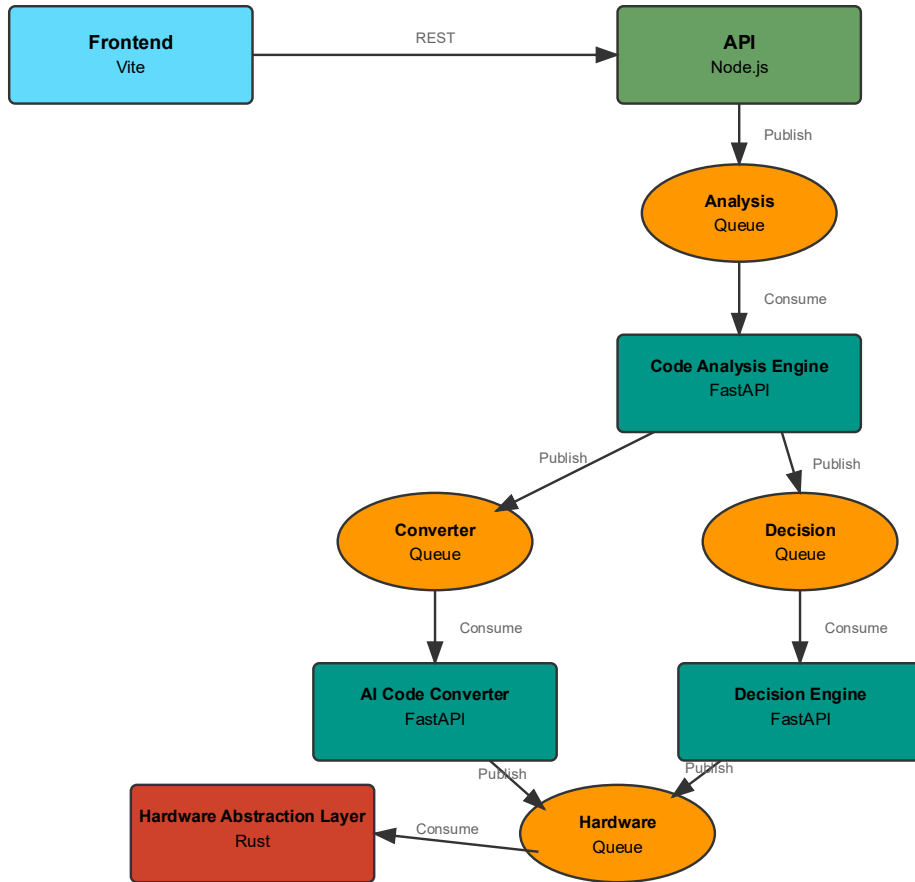


Figure 2.9: Full System Overview Diagram of the Nexar QCCR

Code Analysis Engine occupies the second position in the Nexar pipeline. User code enters the system through the frontend, is passed to the CAE via the TypeScript API gateway, and the CAE's structured analysis output is consumed by the Decision Engine to make routing decisions. The Hardware Abstraction Layer then routes the workload to either quantum hardware (IBM Quantum), classical hardware, or a hybrid execution environment.

2.5 Project Requirements That Have Been Achieved

2.5.1 Functional Requirements

The following functional requirements from the proposal were fully or substantially implemented in the final system.

Table 2.5: Functional Requirement Summary

ID	Requirement	Description
REQ-F-001	Multi-language parsing	The system successfully parses and analyses code from Python/Qiskit, Cirq, OpenQASM, and Q#. Python AST parsing is applied to

		Python-based code, and structured signature/regex parsing is applied to OpenQASM and Q#.
REQ-F-006 – REQ-F-008	Quantum operation detection	Standard quantum gates, qubit declarations, and measurement operations are detected across all four supported frameworks. The gate detection database covers all standard single-qubit and multi-qubit gates from each framework.
REQ-F-011 – REQ-F-014	Complexity analysis	Cyclomatic complexity, line count, function count, and quantum resource metrics including gate counts, circuit depth estimates, and qubit estimates are all computed and included in the analysis response.
REQ-F-016 & REQ-F-018	Algorithm recognition with confidence scores	The single-algorithm classifier identifies ten quantum algorithm types with confidence scores. The multi-label classifier was attempted but did not achieve satisfactory performance, as documented.
REQ-F-020	Analysis report generation	The system generates a structured JSON analysis report containing all computed metrics, which is returned to the caller and displayed in the frontend.

2.5.2 Non-Functional Requirements

Table 2.6: Non-Functional Requirements Summary

ID	Requirement	Description
REQ-NF-003	85% accuracy for known quantum algorithm patterns	The ensemble language classifier achieved 98.52% accuracy, well above the 85% threshold. The single-algorithm classifier achieved 86.39% accuracy, narrowly exceeding the threshold. The multi-label classifier did not meet this threshold and is documented as a limitation.
REQ-NF-005	System availability	The CAE is implemented as a stable FastAPI service with standard error handling and request validation, maintaining the uptime required for integration with the broader Nexar system during development and testing.
REQ-NF-009 & REQ-NF-011	Modular architecture	The CAE's modular design — with language classification, algorithm classification, classical complexity, and quantum analysis as separate, independently testable modules — satisfies the requirement for a modular architecture that enables independent component upgrades.

2.5.3 Other Requirements

API Integration

The CAE exposes a well-defined RESTful API with JSON request and response bodies, CORS support, and automatic OpenAPI documentation via FastAPI's built-in swagger UI at `/docs`.

Frontend Integration

The CAE is fully integrated with the React frontend, providing both real-time language detection and full analysis result visualisation.

Model Training Infrastructure

All trained models are persisted to the `models/trained/` directory and loaded at service startup, avoiding the need to retrain models on each service restart.

2.6 Consideration of Aspects of the System

2.6.1 Social Aspects

The Code Analysis Engine is designed to lower the barrier to entry for quantum computing by removing the need for deep language-specific expertise to characterise quantum code. Developers who are proficient in one quantum framework but unfamiliar with others can submit code for analysis without needing to understand the syntactic conventions of the target framework. This democratisation of quantum code analysis has a positive social impact by making quantum computing more accessible to a broader developer population.

The broader Nexar system of which the CAE is a part aims to make quantum cloud computing more cost-efficient by routing workloads intelligently. Reducing unnecessary quantum hardware usage has an indirect environmental benefit, as quantum hardware requires significant energy for cooling and operation.

2.6.2 Security Aspects

The CAE handles submitted source code, which may be proprietary or commercially sensitive. The current implementation does not store submitted code persistently — code is analysed in memory and the results are returned, with no logging of the code content itself. This is a deliberate design choice to minimise security risks.

The REST API is currently designed for internal use within the Nexar system (accessed through the API gateway) rather than direct public exposure. API endpoint authentication and HTTPS encryption would be required before deploying the CAE in a production environment where the service is exposed to external networks.

The machine learning models do not learn from submitted code at inference time — they use fixed, pre-trained weights. There is therefore no risk of model poisoning through carefully crafted inference requests.

2.6.3 Ethical Aspects

The training data for the machine learning models was collected from public open-source repositories on GitHub and from official framework documentation and tutorials. No private or proprietary code was used in training. The use of publicly licensed open-source code for academic machine learning research is consistent with standard practices in the field.

The system makes probabilistic predictions with explicit confidence scores rather than presenting its classifications as ground truth. Users are therefore informed of the uncertainty in the system's predictions, which is ethically important in a context where incorrect classifications could lead to incorrect routing decisions with associated computational and financial costs.

The multi-label classifier's limitations are disclosed fully in this report. It would be unethical to deploy a classifier that performs poorly on some classes without disclosing this limitation, and the decision not to integrate the multi-label classifier into the production system reflects this ethical consideration.

2.6.4 Limitations

Several important limitations of the CAE as implemented must be acknowledged honestly.

Multi-label algorithm classification failure

The experimental multi-label CodeBERT classifier did not achieve satisfactory performance, primarily due to severe dataset imbalance caused by the uneven availability of multi-algorithm quantum code implementations across different frameworks. This remains an open problem for future work.

Regex-based gate detection semantic gap

The current quantum analysis pipeline uses regex and simple AST-based gate detection that cannot fully resolve the semantic meaning of detected gates. Gates inside loops are not automatically unrolled. Dynamically computed qubit indices are not always resolvable. This means that circuit depth and qubit count estimates may be inaccurate for complex, dynamically constructed circuits.

Limited Q# and OpenQASM parsing depth

The CAE's deepest parsing capabilities are for Python-based Qiskit and Cirq code, as these benefit from the full Python AST. For Q# and OpenQASM, the system relies on signature matching and regex patterns, which provide useful but less structurally complete analysis.

Single-algorithm assumption

The single-algorithm classifier is designed under the assumption that a code snippet implements one dominant algorithm. Real-world quantum programs often compose multiple algorithms as sub-routines, and the single-algorithm classifier cannot handle this correctly. The multi-label classifier was intended to address this but did not achieve production-ready performance.

Dataset scale

All training data for the ML classifiers was manually collected or synthesised in the context of an individual research project with limited resources. Larger, more diverse datasets would almost certainly improve classification performance, particularly for the algorithm classifier.

Canonical IR incomplete implementation

The Canonical IR enhancement was partially implemented and demonstrated on simple cases. A full, production-grade implementation would require substantially more engineering effort and would need to handle all edge cases in the construction of quantum circuits across all four supported frameworks.

2.7 Commercialisation Aspects of the Product

2.7.1 Target Audience

The primary target audience for the Code Analysis Engine consists of software development teams and research groups actively working in hybrid quantum-classical computing. This includes research institutions experimenting with quantum algorithms on IBM Quantum, Google Quantum AI, or Microsoft Azure Quantum; software companies building quantum-accelerated applications; and academic teams conducting quantum software engineering research. A secondary audience includes quantum computing educators who could use the system's analysis capabilities to provide students with structured feedback on their quantum code.

2.7.2 Market Space

The quantum computing software market is growing rapidly alongside the expansion of quantum cloud services offered by IBM, Google, Microsoft, and Amazon [1, 4]. As more organisations experiment with quantum computing, there is a growing need for developer tooling that reduces the complexity of working with quantum code. The Code Analysis Engine occupies a niche that no existing commercial product currently fills: a multi-language, ML-powered quantum code analysis service.

The CAE could be positioned as a SaaS product offered as an add-on to quantum cloud platforms, as an IDE extension for VS Code or JetBrains, or as an enterprise API service for organisations building internal quantum software pipelines.

2.7.3 Revenue Earning

Potential revenue models include a freemium API model with usage-based pricing for the analysis service, enterprise licensing to quantum software companies that want to embed the analysis capability in their own tooling, and academic licensing to universities and research institutions for educational use. The modular architecture of the CAE makes it straightforward to package and distribute as a standalone service.

2.8 Testing and Implementation

2.8.1 Code Implementation of the Research Part

The following describes the actual code implementation of each major component of the Code Analysis Engine.

Service Entry Point

The CAE is implemented as a Python FastAPI application. The `main.py` file defines the application, registers the API endpoints, configures CORS middleware, and loads the trained ML models at startup. The primary analysis endpoint is defined at `POST /analyse`, which accepts a JSON body containing a code field and returns a structured JSON analysis report.

Language Classifier Implementation

The language classifier is implemented as a Python class (`LanguageClassifier`) that encapsulates all four base models and the ensemble logic. During training, each model is trained on the dataset using the feature extraction pipeline described in Section 2.2.3. The trained models and TF-IDF vectoriser are serialised using `joblib` (for tree-based models) and HuggingFace's `save_pretrained` mechanism (for CodeBERT) and stored in the `models/trained/language_classifier/` directory.

At inference time, the `LanguageClassifier.predict()` method processes the incoming code through all four models, collects their probability outputs, applies the configured weights, and returns the predicted class label along with the confidence score. The implementation handles edge cases including empty code strings, code that triggers syntax errors in the feature extractor, and code that contains no recognisable quantum patterns.

The training script for the language classifier also produces a comprehensive evaluation report including per-class precision, recall, and F1 scores, a confusion matrix, and per-model individual accuracy scores. These artifacts are stored alongside the models for reference.

Feature Extraction for Tree-Based Models

The feature extraction pipeline for the tree-based classifiers (Random Forest, Gradient Boosting, XGBoost) is implemented as a `QuantumCodeFeatureExtractor` class. This class applies a fitted TF-IDF vectoriser to the raw source code, producing a sparse feature matrix, and optionally concatenates handcrafted signature score vectors. The TF-IDF vectoriser is configured with a maximum vocabulary size, character-level n-grams in addition to word-level tokens, and sublinear TF weighting to mitigate the effect of very common tokens.

The handcrafted signature detector counts the occurrence of framework-specific patterns in the code text. For Qiskit, these include `QuantumCircuit`, `qc.h()`, `qc.cx()`, `from qiskit import`, and similar. For Cirq, they include `cirq.`, `cirq.LineQubit`, `cirq.Circuit`. For Q#, they include `operation`, `function`, `using`, `Qubit`, and `Microsoft.Quantum`. For OpenQASM, they include `OPENQASM`, `qreg`, `creg`, `gate`. Each signature match contributes a weighted score to the corresponding class, and these scores form an additional feature vector.

CodeBERT Fine-Tuning

The CodeBERT component of both the language and algorithm classifiers tuned using the HuggingFace transformers library. The `RobertaForSequenceClassification` model with the `microsoft/codebert-base` pre-trained weights was used as the base. A linear classification head with the appropriate number of output classes (5 for language, 10

for algorithm) was added on top of the [CLS] token embedding. Training used the AdamW optimiser with a cosine learning rate schedule.

For each code sample, the input was constructed by tokenising the raw source code directly, truncating to 512 tokens if necessary. No special prompt engineering or code preprocessing (such as comment stripping or normalisation) was applied, as the raw code format including comments and docstrings was found to be informative for classification.

Single-Algorithm Classifier Implementation

The single-algorithm classifier is implemented using the same EnsembleClassifier base class as the language classifier, with the dataset and the number of output classes as the only structural differences. The training script for the algorithm classifier includes algorithm-specific data loading, the same feature extraction pipeline, and the same ensemble combination logic.

A key engineering detail of the algorithm classifier is that its training dataset has significantly larger class imbalance than the language classifier dataset. To mitigate this, class weights were incorporated into the tree-based model training classes with fewer samples received higher weights in the loss function, encouraging the models to perform better on under-represented classes. CodeBERT's fine-tuning also used weighted sampling to ensure that each batch contained a balanced mix of algorithm classes.

Multi-Label Classifier Implementation

The multi-label classifier is implemented as a separate module using a BertForMultiLabelClassification class that wraps the microsoft/codebert-base model with a sigmoid-activated multi-label head. The forward pass produces logits of shape (batch_size, num_labels), which are passed through a sigmoid function to produce per-class probabilities. Binary cross-entropy loss is computed independently for each label position.

The training loop for the multi-label classifier evaluates Hamming loss, exact match accuracy, and per-class F1 scores at each epoch using the validation set. Model checkpoints are saved for the epoch that achieves the best F1-micro score on the validation set. The best model was achieved at epoch 14 out of 15, suggesting that additional training epochs would not have substantially improved performance given the dataset limitations.

Classical Complexity Analyser

The classical complexity analyser is implemented as a Python ast.NodeVisitor subclass. Upon instantiation, it receives the source code string and calls ast.parse() to produce the AST. The visit_* methods are implemented for each relevant node type: visit_FunctionDef increments the function counter and records the function name; visit_ClassDef increments the class counter; visit_If, visit_For, visit_While, visit_Try, and visit_ExceptHandler each increment the cyclomatic complexity counter; visit_Import and visit_ImportFrom increment the import counter.

The total cyclomatic complexity is computed as the sum of branch points across all functions plus 1 per function (the baseline). The analyser returns a structured result object containing all collected metrics.

Quantum Analysis Pipeline

The quantum analysis pipeline is implemented as a chain of framework-specific analyser classes. The appropriate analyser is selected based on the output of the language classifier. For Qiskit code, the `QiskitAnalyser` class is applied; for Cirq, the `CirqAnalyser`; and so on.

Each analyser class implements the same interface: a `detect_gates()` method, a `estimate_qubits()` method, a `estimate_depth()` method, and a `detect_entanglement()` method. The internal implementations differ by framework: the Qiskit analyser uses Python AST traversal to identify `QuantumCircuit` method calls, while the OpenQASM analyser uses regular expressions to match assembly mnemonics.

The circuit depth estimation is implemented using a dependency graph approach. Gates are modelled as nodes, and for each qubit, directed edges are added from each gate to the next gate that uses the same qubit. The depth is then the length of the longest path through this directed acyclic graph, computed using a dynamic programming approach.

Canonical IR Implementation

The Canonical IR was implemented as a data class hierarchy with `CanonicalCircuit`, `GateOperation`, `QubitReference`, and `ExecutionContext` as the primary types. A `CanonicalIRConverter` class was implemented for Qiskit code, which uses Qiskit's own transpiler and circuit representation internally to produce a reliable gate-level description. This converter resolves qubit indices by evaluating the circuit's qubit register assignments and mapping gate arguments to specific qubit objects.

The `ExecutionContext` object attached to each `GateOperation` records whether the gate is inside a for loop (and with what range), whether it is inside an if conditional, and whether it is inside a function that is called multiple times. For statically analysable loops (those with integer range bounds), the gate's actual execution count can be computed exactly.

API Response Schema

The analysis response is defined as a Pydantic model with nested fields for language detection results, algorithm detection results, classical complexity metrics, and quantum analysis metrics. Each field includes appropriate type annotations and default values for cases where analysis could not be completed. The response schema is documented automatically in the FastAPI-generated OpenAPI specification.

2.8.2 Testing

Unit Testing

Unit tests were written for each major module of the CAE using the `pytest` framework. Tests covered the feature extraction pipeline (verifying that the TF-IDF vectoriser produces outputs of the expected dimensions and that signature scores are

correctly computed), the classical complexity analyser (verifying that cyclomatic complexity counts are correct for known code snippets), the quantum gate detector (verifying that specific gate patterns are detected correctly in synthesised code samples), and the API endpoint (verifying that the response schema is correct and that appropriate error responses are returned for invalid inputs).

Model Evaluation

Each trained model was evaluated on a held-out test set that was not used during training or hyperparameter selection. The evaluation metrics reported include accuracy, precision, recall, and F1 score at both the macro and weighted levels, as well as a per-class breakdown and confusion matrix. The evaluation was scripted as a standalone evaluation harness that loads the saved model artifacts and runs inference on the test set, producing both human-readable text reports and JSON evaluation summaries.

For the language classifier, the evaluation set comprised 1,080 samples across five classes. For the single-algorithm classifier, it comprised 8,800 samples across ten classes. For the multi-label classifier, the evaluation set comprised 750 samples across eight classes.

Integration Testing

Integration tests verified the end-to-end behaviour of the analysis pipeline by submitting sample code strings to the running FastAPI service and verifying that the returned JSON response contained all expected fields and that the predicted language and algorithm matched the known ground truth for the submitted samples. Integration tests were run against both Qiskit and Cirq code samples, as well as Q# and OpenQASM samples.

Frontend Testing

The frontend integration was tested manually by submitting code samples through the browser interface and verifying that the real-time language detection, the analysis request, and the results display all functioned correctly. This included testing error states — for example, submitting empty code, submitting code that results in a parsing error, and submitting code for which no quantum gates could be detected.

Cross-Framework Testing

A specific testing effort was directed at verifying cross-framework consistency that implementations of the same quantum algorithm in different frameworks (e.g., a Grover's search implementation in Qiskit vs. in Cirq) were correctly classified as the same algorithm. This required careful selection of test samples that represented the same algorithm in both frameworks and verified that the single-algorithm classifier produced consistent predictions.

2.9 Tools and Technologies

Table 2.7: Tools and Technologies Used

Category	Tool / Technology	Version / Notes
Language	Python	3.10+
Web Framework	FastAPI	Latest stable
ML Framework	HuggingFace Transformers	Latest stable
ML Framework	Scikit-learn	Latest stable
ML Framework	XGBoost	Latest stable
Pre-trained Model	microsoft/codebert-base	RoBERTa-based
Code Analysis	Python ast module	Native
Feature Extraction	TF-IDF (Scikit-learn)	
Data Handling	NumPy, Pandas	
Model Serialisation	joblib, HuggingFace save_pretrained	
Quantum Framework	IBM Qiskit	For Canonical IR
Frontend	React + Vite	TypeScript
API Gateway	TypeScript Express	Port 3000
Version Control	Git / GitHub	
Containerisation	Docker	For deployment
API Documentation	FastAPI / OpenAPI	Auto-generated
Testing	pytest	
Platform	Ubuntu 20.04 LTS	Development / deployment

3 RESULTS AND DISCUSSION

3.1 Results

3.1.1 Language Classifier Results

The ensemble language classifier was evaluated on a test set of 1,080 samples across five classes (Cirq, OpenQASM, Python, Qiskit, Q#). The evaluation was performed on a held-out test split that was not seen during training or validation.

Individual Model Performance

Before presenting ensemble results, the individual performance of each component model provides insight into the strengths of the ensemble architecture. As shown in Table 3.1, all four models achieved strong individual performance, with XGBoost and the ensemble tied for the highest overall accuracy at 98.52%.

Table 3.1: Language Classifier — Accuracy by Model

Model	Accuracy
CodeBERT	98.33%
XGBoost	98.52%
Random Forest	97.22%
Gradient Boosting	98.24%
Ensemble	98.52%

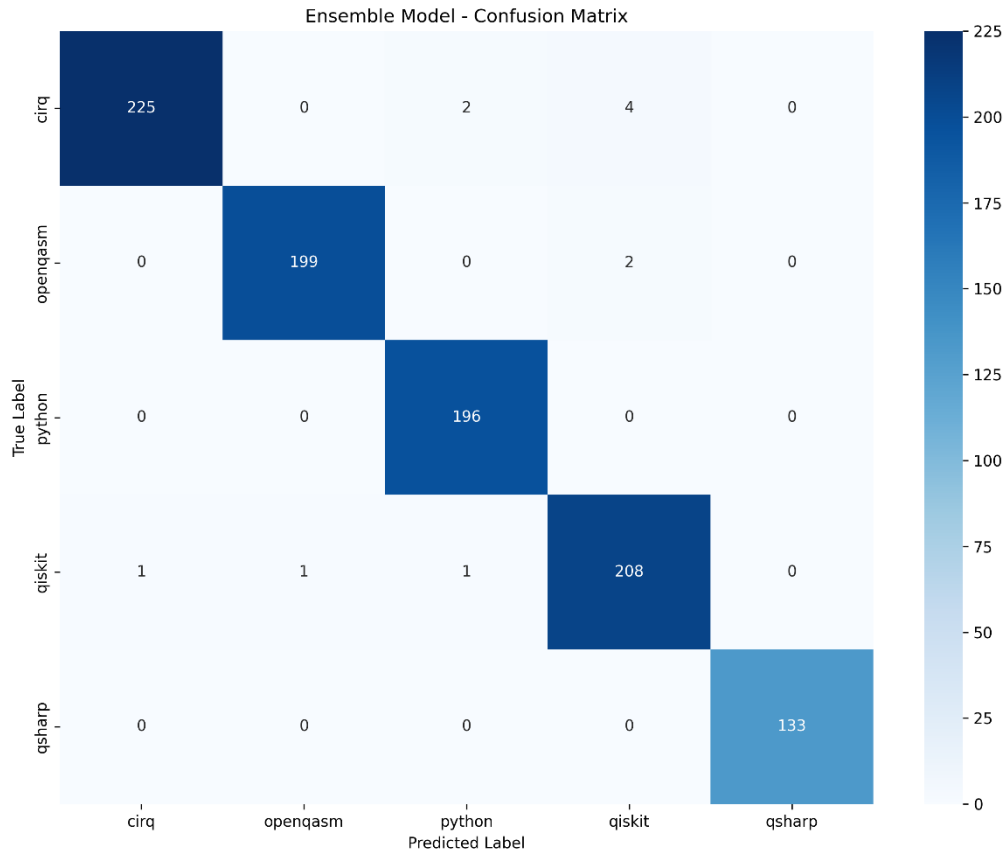


Figure 3.1: Ensemble Language Classifier — Training Confusion Matrices (All Models)

Ensemble Performance

The ensemble achieved an overall accuracy of 98.52%, a macro-averaged precision of 98.59%, a macro-averaged recall of 98.66%, and a macro-averaged F1 score of 98.62%. Table 3.2 presents the complete ensemble metrics.

Table 3.2: Language Classifier — Ensemble Metrics (Test Set)

Metric	Value
Accuracy	98.52%
Precision (Macro)	98.59%
Recall (Macro)	98.66%
F1 (Macro)	98.62%
Precision (Weighted)	98.53%
Recall (Weighted)	98.52%
F1 (Weighted)	98.52%

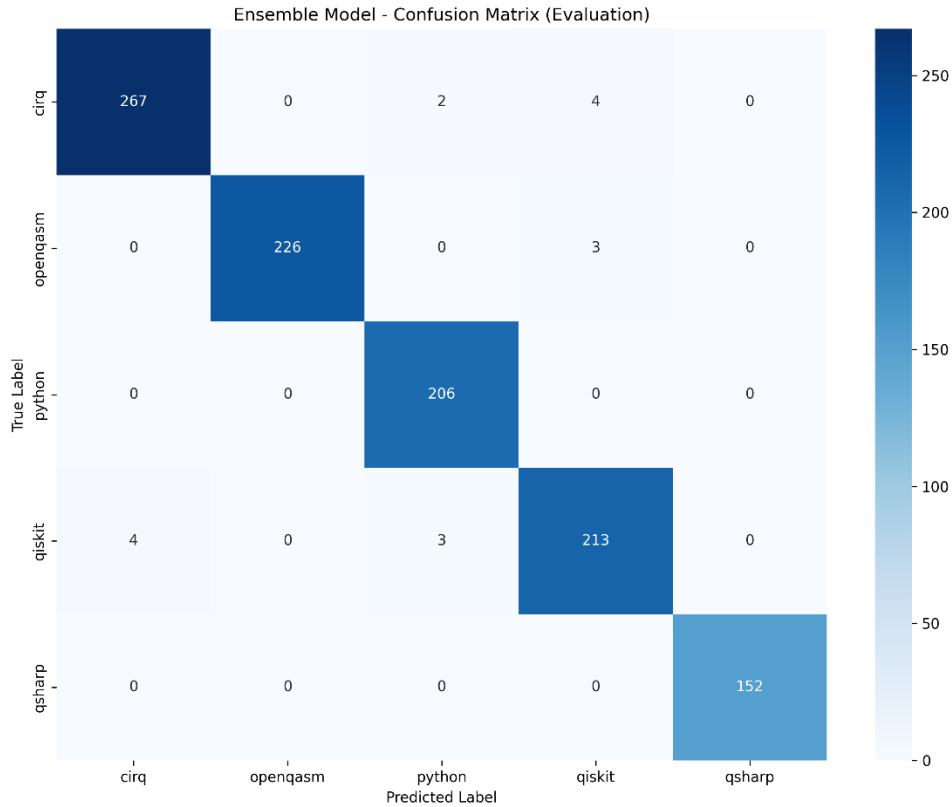


Figure 3.2: Ensemble Language Classifier — Evaluation Confusion Matrix

Per-Class Performance

The per-class breakdown reveals that Q# achieved perfect precision and recall ($F1 = 1.0$), reflecting the highly distinctive syntactic signatures of the Q# language. OpenQASM also achieved near-perfect precision (1.0) with slightly lower recall (0.987). Python achieved perfect recall (1.0) with slightly lower precision (0.976). Qiskit showed the lowest performance of the five classes, with precision and recall both at 0.968 , reflecting the inherent challenge in distinguishing Qiskit code from generic Python code a challenge that was anticipated during dataset construction given that Qiskit code is syntactically valid Python.

Table 3.3: Language Classifier — Per-Class Classification Report

Class	Precision	Recall	F1-Score	Support
Cirq	98.52%	97.80%	98.16%	273
OpenQASM	100.00%	98.69%	99.34%	229
Python	97.63%	100.00%	98.80%	206
Qiskit	96.82%	96.82%	96.82%	220
Q#	100.00%	100.00%	100.00%	152

The confusion matrix (Figure 3.2) shows that the primary confusion source is between Cirq and Qiskit (4 Cirq samples misclassified as Qiskit and 4 Qiskit samples misclassified as Cirq), and between OpenQASM and Qiskit (3 OpenQASM samples misclassified as Qiskit). This is consistent with expectations — Cirq and Qiskit both use Python-like syntax, and some OpenQASM constructs resemble the naming conventions used in Qiskit.

3.1.2 Single-Algorithm Classifier Results

The single-algorithm ensemble classifier was evaluated on a test set of 8,800 samples across ten quantum algorithm classes.

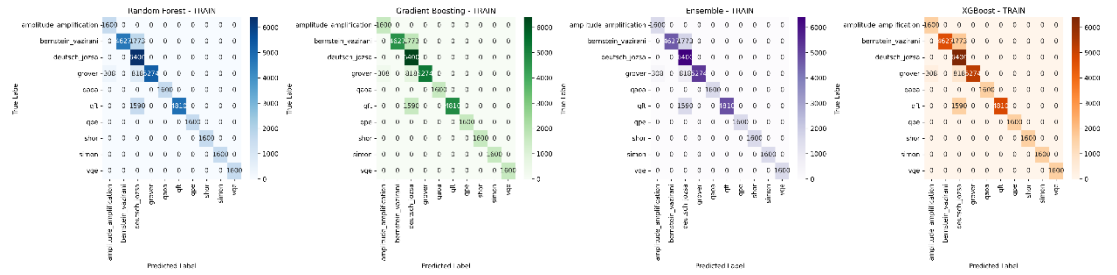


Figure 3.3: Single-Algorithm Classifier — Training Confusion Matrices (All Models)

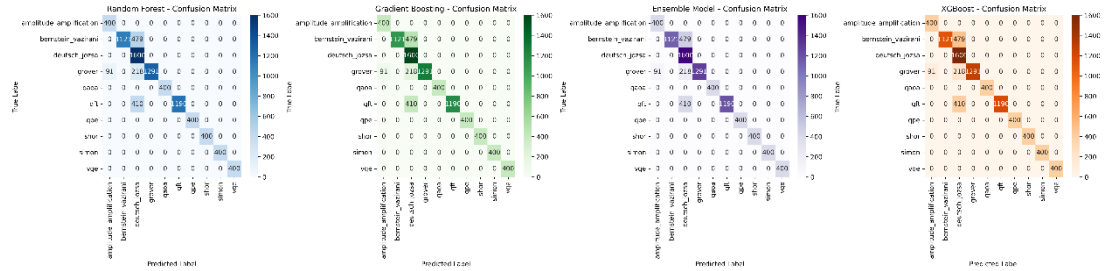


Figure 3.4: Single-Algorithm Classifier — Evaluation Confusion Matrix

Overall Performance

The ensemble achieved an overall accuracy of 86.39%, a macro-averaged precision of 94.06%, a macro-averaged recall of 92.51%, and a macro-averaged F1 score of 92.11%.

Per-Class Performance

Table 3.4 shows the per-class breakdown. Perfect or near-perfect classification was achieved for QAOA, QPE, Shor, Simon, and VQE — all classes with highly distinctive gate patterns and structural signatures. Grover's algorithm achieved a recall of 80.69%, with some samples being misclassified as Amplitude Amplification, which is expected given that Grover's algorithm is a specific instance of the more general amplitude amplification procedure. QFT achieved a recall of 74.38%, with some misclassifications to other classes. Bernstein-Vazirani achieved a recall of 70.06%, likely because its circuit structure (a single layer of CNOT gates followed by a Hadamard layer) overlaps with generic quantum circuits. Deutsch-Jozsa achieved a recall of 100% but a precision of only 59.1%, indicating that many samples from other classes were incorrectly classified as Deutsch-Jozsa — again an expected outcome given the similarity between Deutsch-Jozsa and other oracle-based algorithms.

Table 3.4: Single-Algorithm Classifier — Per-Class Classification Report

Algorithm	Precision	Recall	F1-Score	Support
Amplitude Amplification	81.47%	100.00%	89.79%	400
Bernstein-Vazirani	100.00%	70.06%	82.40%	1600

Deutsch-Jozsa	59.11%	100.00%	74.30%	1600
Grover	100.00%	80.69%	89.31%	1600
QAOA	100.00%	100.00%	100.00%	400
QFT	100.00%	74.38%	85.30%	1600
QPE	100.00%	100.00%	100.00%	400
Shor	100.00%	100.00%	100.00%	400
Simon	100.00%	100.00%	100.00%	400
VQE	100.00%	100.00%	100.00%	400

The pattern of results is consistent with the structural relationships between quantum algorithms. Algorithms with highly distinctive and unique gate sequences (QAOA, QPE, Shor, Simon, VQE) are classified perfectly. Algorithms that share structural similarity with others (Deutsch-Jozsa / Amplitude Amplification / Grover) show lower performance on the metrics most sensitive to inter-class confusion.

3.1.3 Multi-Label Algorithm Classifier Results

The multi-label CodeBERT classifier was evaluated on a test set of 750 samples across eight algorithm classes.

Overall Performance

The best epoch (epoch 14) achieved a Hamming loss of 0.1515, an exact match accuracy of 0.40, an F1-micro of 0.3976, an F1-macro of 0.6635, and an F1-weighted of 0.3446.

Table 3.5: Multi-Label CodeBERT — Best Epoch Metrics

Metric	Best Value (Epoch 14)
Hamming Loss	0.1515
Exact Match Accuracy	0.40
F1 Micro	0.3976
F1 Macro	0.6635
F1 Weighted	0.3446

Per-Algorithm F1 Scores

The extreme variance in per-algorithm F1 scores clearly demonstrates the impact of dataset imbalance.

Table 3.6: Multi-Label CodeBERT — Per-Algorithm F1 Scores

Algorithm	F1 Score
Bernstein-Vazirani	0.000
Deutsch-Jozsa	0.000
Grover	1.000
QAOA	1.000
QFT	0.839
QPE	1.000

Shor	1.000
Simon	0.469

These results confirm that the multi-label classifier's poor overall performance is entirely attributable to the failure to learn the minority classes (Bernstein-Vazirani with 600 training samples but structured as multi-label caused confusion, and Deutsch-Jozsa similarly). The model learned to classify Grover, QAOA, QPE, and Shor perfectly because these had well-defined, balanced, and structurally distinctive training samples in the multi-label context. Simon was partially learned ($F1 = 0.469$). Bernstein-Vazirani and Deutsch-Jozsa received zero F1 because the model learned to predict them as negative for all inputs — a rational strategy that minimises loss when the positive class is rare and ambiguous.

3.2 Research Findings and Discussion

Finding 1: Ensemble combining CodeBERT and tree-based models is highly effective for quantum language classification

The 98.52% accuracy achieved by the language classifier demonstrates that combining a transformer-based model's deep semantic representations with the complementary strengths of tree-based ensemble methods produces a classifier that is robust, accurate, and reliable. This finding has practical significance: language classification is the first step in the CAE's analysis pipeline, and any errors at this stage propagate to all downstream components. The near-perfect language classification performance means that the vast majority of subsequent analysis is performed by the correct framework-specific parser and analyser.

The use of ensemble learning to combine fundamentally different model families a transformer and tree-based models is a methodological contribution of this research. Prior work on code classification has typically used either neural approaches or traditional machine learning, not both in combination. The results here suggest that both paradigms offer complementary information and that their combination is beneficial.

Finding 2: Single-algorithm quantum algorithm recognition is achievable with over 86% accuracy using relatively small datasets

The 86.39% accuracy of the single-algorithm classifier is a meaningful result given the relatively small size of the training dataset and the fact that it was manually constructed without large-scale data collection infrastructure. This demonstrates that quantum algorithm recognition is a learnable problem and that machine learning can extract meaningful structural patterns from quantum code.

The algorithms that are classified most accurately (QAOA, QPE, Shor, Simon, VQE) tend to have highly distinctive gate patterns: QAOA uses parameterised two-qubit cost Hamiltonians that are structurally unlike other algorithms; Shor's algorithm involves modular exponentiation circuits; Simon's algorithm uses a specific oracle structure. Algorithms that are classified with lower accuracy (Deutsch-Jozsa, Bernstein-

Vazirani, QFT) are those whose structural signatures overlap more with other algorithms or are simpler circuits with fewer distinguishing features. These observations are consistent with the theoretical understanding of these algorithms.

Finding 3: Multi-label algorithm classification requires substantially larger and better-balanced datasets than are currently available

This is perhaps the most important research finding from the perspective of the broader field. The failure of the multi-label classifier is not a failure of the model architecture CodeBERT is well-suited to this task but a failure of data. The quantum computing ecosystem does not yet have a large enough body of publicly available, labelled, multi-algorithm quantum code to train a reliable multi-label classifier. This is analogous to the early days of natural language processing, where the unavailability of large annotated corpora was the primary bottleneck before the creation of datasets like ImageNet, SQuAD, and GLUE enabled the deep learning revolution in those fields. The quantum software engineering community needs equivalent benchmark datasets to make progress on problems like multi-label algorithm recognition.

Finding 4: The Unified AST approach loses critical semantic information for quantum analysis

The analysis of the implementation's limitations revealed that a flat list-based representation of detected quantum operations is fundamentally insufficient for computing accurate quantum metrics. The Canonical IR approach, which preserves qubit connectivity, data flow dependencies, and execution context, is the correct direction for more accurate quantum analysis. This finding aligns with the trajectory of the broader programming language research community, which has moved from syntactic to semantic representations for increasingly sophisticated code analysis tasks.

Finding 5: Real-time language detection on the frontend is both feasible and valuable

The implementation of a JavaScript-based language detection system on the frontend mirroring the Python backend logic provides immediate visual feedback to users without requiring a server round trip. This user experience improvement demonstrates that the signature-based component of the language classification approach is lightweight enough to run in a browser in real time. The combination of fast frontend detection for immediate feedback and the more accurate ensemble backend for definitive classification provides a good balance between responsiveness and accuracy.

Discussion: Relationship between dataset quality and model performance

The most striking pattern across all three classifiers is the direct relationship between the quality and balance of the training dataset and the model's performance on specific classes. Where data was plentiful and balanced, performance was near-perfect (Q# language classification, QAOA algorithm classification). Where data was sparse or imbalanced, performance degraded dramatically (multi-label Bernstein-Vazirani and Deutsch-Jozsa). This underscores a fundamental challenge for quantum code analysis research: the practical limitation imposed by the current state of the quantum software ecosystem, where large, diverse, labelled quantum code repositories do not yet exist.

Future work should invest substantially in dataset construction as a prerequisite for further progress.

Discussion: Practical implications for workload routing

The CAE's outputs are consumed by the Nexar Decision Engine to make routing decisions. From this perspective, the language classification result (98.52% accurate) is highly reliable and can be used with confidence. The single-algorithm classification result (86.39% accurate) is useful but should be supplemented with confidence score thresholding only routing decisions that rely on algorithm classification should be made with confidence if the classifier's confidence score exceeds a meaningful threshold. The multi-label classification result should not be used in the current production system due to its poor performance.

Discussion: Comparison with the research gap analysis

The research gap identified in Section 1.2 described the absence of multi-language ML-based quantum code analysis. The CAE directly addresses this gap by providing both language and algorithm classification across four quantum frameworks. The research has demonstrated that this is technically feasible and practically achievable within the resource constraints of an individual research project. At the same time, the multi-label experiment has revealed a new, deeper gap: the absence of adequate datasets for multi-algorithm detection, which represents the next frontier for this line of research.

4 CONCLUSION

This research has successfully designed, implemented, and evaluated the Code Analysis Engine (CAE) as a core component of the Nexar Quantum-Classical Code Router system. The CAE addresses the critical challenge of automated quantum code characterisation in hybrid computing environments by combining machine learning-based classification with traditional static analysis techniques.

The primary contributions of this research are as follows. A multi-model ensemble language classifier was developed that combines CodeBERT, Random Forest, Gradient Boosting, and XGBoost to identify quantum programming languages and frameworks with 98.52% accuracy across five classes. A single-algorithm quantum algorithm classifier was developed using the same ensemble strategy, achieving 86.39% accuracy across ten quantum algorithm classes on a test set of 8,800 samples. An experimental multi-label algorithm classifier was implemented using fine-tuned CodeBERT, which revealed fundamental dataset limitations that currently prevent reliable multi-label quantum algorithm detection a finding that itself represents an important contribution to the field. A classical complexity analysis module was implemented using Python's native AST module. A quantum analysis pipeline was developed and integrated, providing gate detection, qubit estimation, and circuit depth estimation across four quantum frameworks. The Canonical IR enhancement concept was explored and partially implemented as a direction towards more semantically accurate quantum code analysis.

The system has been integrated as a FastAPI microservice within the broader six-service Nexar system, with a React frontend providing real-time language detection and analysis result visualisation.

The most honest assessment of what was achieved is this: language and single-algorithm classification are solved at a practical level by this research. Multi-label algorithm detection remains an open problem, and the investigation of this problem has clarified why it is hard and what would be needed to solve it. The quantum-specific analysis pipeline provides useful approximate metrics but requires more sophisticated parsing along the lines of the Canonical IR approach for production-grade accuracy.

Looking ahead, future work should focus on several directions. The multi-label algorithm classifier requires a substantially larger and more carefully balanced training dataset, ideally collected from production quantum software repositories rather than manually authored samples. The Canonical IR implementation should be completed to enable deterministic, semantically correct quantum metric extraction. The system's support should be extended to additional quantum frameworks including PennyLane, Braket SDK, and newer versions of OpenQASM. Finally, the language and algorithm classifiers could be refined with active learning approaches that use human-expert feedback to iteratively improve performance on difficult or ambiguous cases.

The Nexar project as a whole, of which the CAE is one component, demonstrates the feasibility of building intelligent hybrid quantum-classical workload orchestration infrastructure. As quantum cloud services become increasingly accessible and quantum hardware matures toward fault tolerance, systems like Nexar and the code

analysis capabilities at their core will become essential infrastructure for practical quantum software development.

5 REFERENCES

- [1] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.
- [2] A. JavadiAbhari, S. Patil, D. Kudrow, J. Heckey, A. Lvov, F. T. Chong, and M. Martonosi, "ScaffCC: a framework for compilation and analysis of quantum computing programs," in *Proceedings of the 11th ACM Conference on Computing Frontiers (CF '14)*, New York, NY, USA: Association for Computing Machinery, 2014, pp. 1–10. doi: 10.1145/2597917.2597939.
- [3] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, "Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 1015–1029.
- [4] A. Glos, A. Krawiec, and Z. Zimborás, "Space-efficient binary optimization for variational quantum computing," *npj Quantum Information*, vol. 8, no. 39, Apr. 2022. doi: 10.1038/s41534-022-00546-y.
- [5] M. Schuld, I. Sinayskiy, and F. Petruccione, "An introduction to quantum machine learning," *Contemporary Physics*, vol. 56, no. 2, 2015.
- [6] P. Zhao, X. Wu, Z. Li, and J. Zhao, "QChecker: Detecting Bugs in Quantum Programs via Static Analysis," *2023 IEEE/ACM 4th International Workshop on Quantum Software Engineering (Q-SE)*. doi: 10.1109/Q-SE59154.2023.00014.
- [7] A. JavadiAbhari et al., "ScaffCC: Scalable compilation and analysis of quantum programs," *Parallel Computing*, vol. 45, pp. 2–17, 2015.
- [8] Lenovo, "AST: Hierarchical Code Structure for Analysis," Lenovo US. [Online]. Available: <https://www.lenovo.com/us/en/glossary/abstract-syntax-tree/>.
- [9] T. Alexander, N. Kanazawa, D. J. Egger, L. Capelluto, C. J. Wood, A. Javadi-Abhari, and D. C. McKay, "Qiskit Pulse: programming quantum computers through the cloud with pulses, gates, and circuits," *Quantum Science and Technology*, vol. 5, no. 4, p. 044006, 2020.
- [10] A. Cross, A. W. Javadi-Abhari, T. Alexander, N. de Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta, and B. R. Johnson, "OpenQASM 3: A broader and deeper quantum assembly language," *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–50, 2022.
- [11] P. K. Nayak, K. V. Kher, M. B. Chandra, M. V. P. Rao, and L. Zhang, "Q-PAC: Quantum Program Analysis for Bug Detection," *arXiv preprint arXiv:2311.17705*, 2023. Available: <https://arxiv.org/abs/2311.17705>.
- [12] R. S. Smith, M. J. Curtis, and W. J. Zeng, "A Practical Quantum Instruction Set Architecture," *arXiv preprint arXiv:1608.03355*, Aug. 2016.
- [13] C. Chamberland et al., "Universal quantum computing with twist-free and temporally encoded lattice surgery," *PRX Quantum*, vol. 3, no. 1, p. 010331, 2022.

- [14] E. Pednault et al., "Pareto-Efficient Quantum Circuit Simulation Using Tensor Contraction Deferral," *arXiv preprint arXiv:1710.05867*, Oct. 2017.
- [15] N. Khammassi, G. G. Guerreschi, I. Ashraf, J. W. Hogaboam, C. G. Almudever, and K. Bertels, "cQASM v1.0: Towards a Common Quantum Assembly Language," *arXiv preprint arXiv:1805.09607*, May 2018.
- [16] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, Online: Association for Computational Linguistics, Nov. 2020, pp. 1536–1547. doi: 10.18653/v1/2020.findings-emnlp.139.
- [17] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.
- [18] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001. doi: 10.1214/aos/1013203451.
- [19] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, New York, NY, USA: ACM, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.